

# Robot autónomo de aproximación y evasión basado en visión

Resumen completo del trabajo: arquitectura, archivos, parámetros y todo lo que se midió hasta hoy

---

Federico Morán

Estado al 2 de octubre de 2026

TPF — IA aplicada a la Robótica  
Maestría en Inteligencia Artificial

## Objetivo

Tener en un solo lugar **todo el trabajo**: qué hace el robot, cómo está armado, qué hace cada archivo y cada parámetro, y **de dónde salió cada número**. Es un resumen para mí, no para proyectar: tiene más texto que una presentación normal.

Cómo está organizado:

1. **Panorama** — qué hace hoy y qué quedó afuera.
2. **Hardware y arquitectura** — las piezas y cómo se hablan.
3. **Archivo por archivo** — el código del Pi y el firmware del ESP32.
4. **Comportamiento** — la máquina de estados y la ley de control.
5. **Parámetro por parámetro** — qué hace cada uno y por qué vale lo que vale.
6. **Historia** — las 15 jornadas, lo que se midió y lo que falló.
7. **Operación y pendientes** — cómo se usa y qué falta.

Fuentes: `PLAN.md` (el plan y la bitácora), `HARDWARE.md`, `RUNBOOK.md`, los comentarios de `src/config.py` y los registros de `logs/`. Si algo de acá no coincide con esos archivos, mandan ellos.

1. Panorama
2. Hardware
3. Arquitectura
4. El código del Pi, archivo por archivo
5. El firmware del ESP32
6. La máquina de estados
7. La ley de control
8. Parámetro por parámetro
9. Protocolo y seguridad
10. Scripts, registros y herramientas
11. Historia y mediciones
12. Problemas, soluciones y el hilo conductor
13. La jornada final: 1 de octubre, en el piso
14. El 2 de octubre: ultrasónico y buzzer
15. Operación, estado y pendientes

## Panorama

---

Un robot que **recorre una sala, reconoce dos objetos con la cámara y se comporta distinto con cada uno**: se acerca a uno y huye del otro.

### **Objetivo: el oso (teddy bear)**

Un oso **impreso en una hoja A4** ( $\sim 20$  cm). El robot lo busca, se le acerca corrigiendo el rumbo con la cámara y frena a  $\sim 30$  cm.

### **Amenaza: la mochila (backpack)**

Una mochila real. El robot retrocede, gira media vuelta y se aleja. Tiene **prioridad incondicional** sobre todo lo demás.

Todo corre **a bordo**: una Raspberry Pi 5 con una red neuronal (YOLOv8n) decide, y un ESP32 mueve los motores. No hay PC externa, ni teleoperación, ni nube. La notebook sólo sirve para lanzar la corrida por SSH y mirar lo que pasa.

## Qué hace el robot hoy (medido en el piso el 1 y el 2 de octubre)

| Qué ve                         | Qué hace                                                                                                                                                                                        | Estado                             |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Nada                           | Mira quieto $\sim 0,6$ s, gira un paso de $\sim 18^\circ$ y repite. Una vuelta entera tarda $\sim 17$ s; después avanza $\sim 18$ cm para cambiar el punto de vista y vuelve a empezar.         | BUSCAR                             |
| El oso                         | Se queda quieto hasta confirmarlo (3 frames, $\sim 0,3$ s), se acerca centrándolo, frena a $\sim 30$ cm y da un pitido largo. Si lo pierde, lo busca hacia el lado donde lo vio por última vez. | APROXIMAR $\rightarrow$ ENCONTRADO |
| La mochila                     | Se queda quieto hasta confirmarla y, con la alarma sonando, retrocede $\sim 30$ cm girando, sigue girando hasta darle la espalda y avanza $\sim 65$ cm. Después vuelve a buscar.                | HUIR                               |
| Algo a menos de 30 cm adelante | Cuando va a avanzar sin el oso a la vista (huyendo o buscando), no avanza: gira en el lugar hasta tener el camino libre.                                                                        | HUIR o BUSCAR                      |
| Oso y mochila                  | Huye: la amenaza gana siempre, incluso estando detenido frente al oso.                                                                                                                          | HUIR                               |

### Lo que se probó

Dieciséis corridas con motores en dos días: diez el 1 de octubre y seis el 2, ya con el ultrasónico y el buzzer. Hay dos videos vistos desde el robot en media/.

### Lo que no se midió todavía

Las 10 corridas por escenario que piden las métricas, y la autonomía del cargador. Los porcentajes de éxito no existen aún.

| Percepción                     |                       | Movimiento                  |                   |
|--------------------------------|-----------------------|-----------------------------|-------------------|
| Modelo                         | YOLOv8n (COCO)        | Rango de velocidad          | 0,091 a 0,380 m/s |
| Tamaño de inferencia           | 320 px                | Velocidad máx. usada        | 0,25 m/s          |
| FPS del lazo completo          | ~10,6 (~9,9 grabando) | Giro buscando               | pasos de ~18°     |
| Inferencia por frame           | ~88 ms                | Giro en régimen             | 124 a 165°/s      |
| Edad del frame al decidir      | ~150 ms               | Distancia de frenado        | ~30 cm del oso    |
| Alcance con el oso A4          | ~1,3 m                | Asimetría de los motores    | 2,11 veces        |
| Campo visual horizontal        | ~52°                  | Trocha (regla / efectiva)   | 21,5 / 18 cm      |
| Confirmar / perder             | 3 / 5 frames          | Rueda                       | 60 mm de diámetro |
| Seguridad                      |                       | Verificación                |                   |
| Vigencia del comando (Pi)      | 0,40 s                | Verificaciones sin hardware | 79                |
| Watchdog (ESP32)               | 500 ms                | Corridas registradas        | CSV + JSON c/u    |
| Ultrasónico: esquivar / llegar | 30 / 20 cm            |                             |                   |
| Corte de emergencia            | conector de 12 V      |                             |                   |

# Qué entra en esta versión y qué quedó afuera

## Adentro, y funcionando

- Detección con YOLOv8n a bordo, a  $\sim 11$  FPS.
- Máquina de estados con arbitraje: buscar, aproximar, encontrado, huir.
- Control proporcional sobre la imagen.
- ESP32 con protocolo serie, compensación de zona muerta por rueda y watchdog.
- Alimentación portátil con dos fuentes separadas.
- **Ultrasónico** al frente: no avanza contra lo que tenga a menos de 30 cm. **Buzzer**: alarma y llegada.
- Registro de cada corrida y análisis automático.
- 79 verificaciones del comportamiento sin hardware.

El ultrasónico y el buzzer habían quedado afuera el 2026-10-01, por falta de tiempo, y entraron al día siguiente (sección “El 2 de octubre”).

## Afuera: segunda versión

- **Sensores atrás y a los costados**: el robot **retrocede y gira a ciegas**, y una pared muy de costado no la ve.
- **Encoders**: sin odometría ni lazo de velocidad.
- **Frenar a 10 cm**: a esa distancia el oso no entra en el cuadro; haría falta un avance final a ciegas.
- **Cámara CSI** (Arducam): nunca fue vista por el Pi 5; se reemplazó por una webcam USB.

## Cómo se cubren los requisitos del módulo

| Requisito                         | Cómo se cubre                                                                                                                                                               |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Aplicación de IA                  | Red convolucional de detección de objetos (YOLOv8n, preentrenada en COCO) haciendo inferencia en tiempo real sobre hardware embebido.                                       |
| Vehículo autónomo no tripulado    | Robot terrestre de tracción diferencial, con la decisión 100 % a bordo.                                                                                                     |
| Percepción → decisión → actuación | La cadena completa cerrada en el robot físico: cámara, red, máquina de estados, ley de control, enlace serie, motores.                                                      |
| Toma de decisiones                | Máquina de estados con arbitraje explícito de conflictos: huir tiene prioridad sobre aproximarse.                                                                           |
| Fusión de sensores                | Cámara y ultrasónico. La cámara dice qué hacer y hacia dónde; el ultrasónico impide avanzar contra un obstáculo y puede dar por alcanzado el objetivo. Entró el 2026-10-02. |

### Lo que el trabajo demuestra

Integrar **percepción con IA** en un **laço de control físico** que corre en **tiempo real** sobre **hardware real**. La mitad de lo interesante salió de la última parte: motores con zonas muertas y asimetrías, imágenes que se rompen, baterías que no sostienen los 5 V.

### D1 — Un solo microcontrolador de bajo nivel: ESP32

Se descartó usar ESP32 y PSoC5LP a la vez: dos firmwares, dos protocolos y dos cadenas de herramientas duplican la superficie de fallas sin sumar nada. Se programa con PlatformIO, en estilo C.

### D2 — Modelo preentrenado en COCO, sin entrenamiento propio

El ciclo captura → etiquetado → entrenamiento agregaba riesgo de cronograma sin ser el aporte del trabajo, que es **integrar la inferencia en un lazo de control físico**. Se compensa con más profundidad en control, arbitraje y medición.

### D3 — Clases por nombre: teddy bear y backpack

Objetos medianos, compactos y de buen contraste. Las clases se declaran **por nombre** y se resuelven contra `model.names`: si cambia el modelo, falla al arrancar en vez de detectar otra cosa en silencio. **En el piso apareció un matiz**: la mochila real es suitcase para el modelo (ver la sección de la jornada final).

### D4 — Huir tiene prioridad sobre aproximar

Ante las dos clases a la vez, gana la conducta de seguridad. Es la regla clásica de la robótica de comportamientos: los de preservación subsumen a los de objetivo.

### D5 — Encoders como extensión, no como núcleo

El sistema tiene que andar en lazo abierto (PWM directo, calibrado por rueda). Un PID sobre una planta descompensada arranca peor, así que la calibración había que hacerla igual. Los encoders quedaron sin cablear.

### D6 — Confirmación temporal de las detecciones

Una detección aislada no cambia el estado: hacen falta **3 frames seguidos** para confirmar una clase y **5 sin verla** para darla por perdida. Evita que un falso positivo dispare una maniobra.

Y dos decisiones de arquitectura que salieron al implementar: la compensación de zona muerta vive **en el ESP32** y no en el Pi, y las capas que *deciden* no tocan hardware (ver “Arquitectura”).

## Hardware

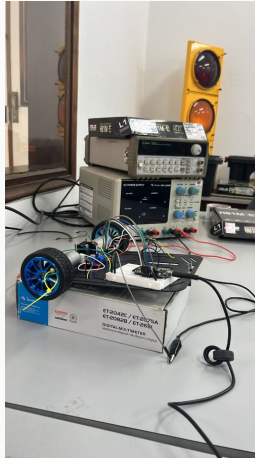
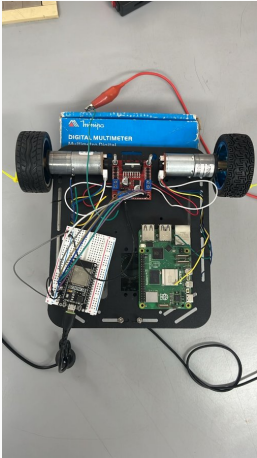
---

## Inventario: qué hay arriba del chasis

| Componente        | Detalle                                              | Rol                                                                  |
|-------------------|------------------------------------------------------|----------------------------------------------------------------------|
| Raspberry Pi 5    | 8 GB de RAM, sin monitor ( <i>headless</i> )         | Alto nivel: cámara, red neuronal, máquina de estados, ley de control |
| ESP32             | DevKit con puente USB CP2102                         | Bajo nivel: PWM de los motores, protocolo serie, watchdog            |
| Driver L298N      | Módulo comercial, puente H doble                     | Etapas de potencia de los dos motores                                |
| Motores (2)       | GM25-370, 12 V, “350 rpm”, con reductora             | Tracción diferencial. <b>Las dos unidades no son iguales</b>         |
| Chasis 2WD        | Dos ruedas de 60 mm y una rueda loca                 | Locomoción                                                           |
| Cámara            | Webcam USB Microsoft LifeCam Studio, 640×480 en MJPG | Dice qué hay y hacia dónde                                           |
| Ultrasónico       | Parallax PING))) , tres pines, al frente             | Distancia a lo que tenga adelante. Desde el 2/10                     |
| Buzzer            | Activo, de dos patas                                 | Alarma al huir y aviso al llegar. Desde el 2/10                      |
| Cargador portátil | 20 000 mAh, 5 V / 3 A, por USB-A                     | Alimenta el Pi (y por sus USB, la webcam y el ESP32)                 |
| UPS               | 10 000 mAh, salida de 12 V                           | Alimenta <b>sólo</b> el L298. Es el corte de emergencia              |
| Encoders Hall     | uno por motor, sin cablear                           | Segunda versión                                                      |

El oso es una **impresión en hoja A4** (~20 cm de ancho) y la mochila es una mochila real, negra. Los dos son parte del sistema: los números de la calibración valen para esos objetos.

## El robot, en el banco (agosto de 2026; el terminado está en “El 2 de octubre”)



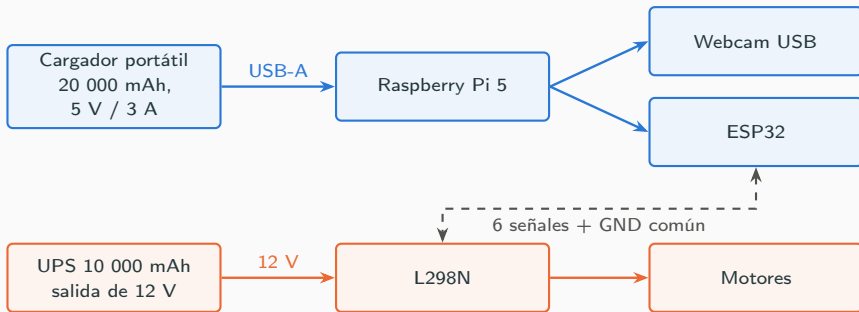
### Qué se ve

- Arriba, los dos motores con sus ruedas y, entre ellos, el módulo **L298N** (rojo).
- A la izquierda, el **ESP32** sobre una protoboard.
- A la derecha, la **Raspberry Pi 5**.
- El robot elevado sobre una caja: así se hicieron todas las pruebas hasta el 30 de septiembre.

### Lo que no se ve

Son fotos del banco. Falta la webcam, que va al frente, y las dos baterías. Las fotos del **robot terminado** están en la sección “El 2 de octubre”.

## Alimentación: dos fuentes separadas



### Por qué separadas

Con la UPS alimentando todo, la salida de 5 V se caía con la carga de YOLO **aun con los motores quietos**: mínimo 4,695 V, el Pi recortando el 43 % del tiempo y el FPS de 11,2 a 8,83. Con el cargador aparte: 11,08 FPS y `get_throttled = 0x0`.

### Tres cosas que no se tocan

- El cargador va por **USB-A**. Por USB-C el Pi entra en bucle de reinicios.
- El **GND común** lo da el cableado ESP32–L298.
- Para cortar los motores se saca el **conector de 12 V**, no la alimentación del Pi.

| L298N       | GPIO              | Función                                    |
|-------------|-------------------|--------------------------------------------|
| ENA         | 25                | PWM del canal A ( <b>rueda derecha</b> )   |
| IN1 / IN2   | 26 / 27           | Sentido del canal A                        |
| ENB         | 13                | PWM del canal B ( <b>rueda izquierda</b> ) |
| IN3 / IN4   | 33 / 32           | Sentido del canal B                        |
| GND         | GND               | Referencia común                           |
| Ultrasónico | 12                | Con divisor 1 k / 2 k                      |
| SIG         |                   |                                            |
| Buzzer      | 14                | Activo, directo al pin                     |
| Encoders    | 34, 35, 36,<br>39 | Reservados; sólo entrada                   |

**El divisor del ultrasónico no es opcional:** el sensor contesta a 5 V y el ESP32 no los tolera. El detalle del cableado está en la sección “El 2 de octubre”.

### PWM

1 kHz y **10 bits** (0 a 1023). Se pasó de 8 a 10 bits para darle resolución a la rueda izquierda, que usa una franja angosta del duty.

### Los canales quedaron cruzados

El canal A del módulo quedó cableado a la rueda **derecha**. No se tocó la bornera: se corrige en el firmware, en el bloque PIN\_LEFT\_\* / PIN\_RIGHT\_\*.

### Jumpers

Los de ENA y ENB van **retirados**: si no, el módulo deja los motores siempre habilitados y el PWM no hace nada.

## Dónde va cada cosa

- **Webcam:** USB azul de abajo (controlador 1).
- **ESP32:** USB negro de abajo (controlador 3), aparece como `/dev/ttyUSB0`.

Quedan en controladores distintos. Compartiendo uno, el ruido de los motores rompía más la imagen.

## Formato de la cámara

**MJPEG**, no YUYV. YUYV ocupa casi todo el ancho de banda del USB 2.0 y no tolera pérdidas: con los motores andando daba 2,3 FPS e imagen en franjas. MJPG transmite  $\sim 10$  veces menos.

## El límite de corriente

Con el cargador portátil el Pi no negocia potencia y limita **todos sus USB a 600 mA**. La webcam declara 500 mA y el ESP32 100 mA: justo el total. Se puede levantar con `usb_max_current_enable=1` (el cargador da 3 A). Está pendiente de probar.

## Efecto colateral del puente USB

Abrir el puerto serie **reinicia el ESP32** (la línea DTR del CP2102). Por eso el enlace espera 2 s al abrir, y por eso leer telemetría “para mirar” frena los motores.

## El L298N: lo que dice la hoja de datos y lo que se midió

### La caída de tensión

El L298 es un puente de transistores bipolares: se queda con parte de la tensión. La hoja de datos sugería  $\sim 1,4$  V; **medido el 2026-07-24: 2,5 V**. Con 12 V de entrada, a los motores les llegan  $\sim 9,5$  V.

### Consecuencia

Fue la primera de las estimaciones que la medición desmintió. A partir de ahí la regla del proyecto: **medir antes de creerle a la hoja de datos**.

### Velocidad máxima real

Con rueda de 60 mm y 9,5 V en bornes: **0,87 m/s** a duty completo. A 5 FPS eso son 17 cm entre un frame y el siguiente: demasiado para controlar con visión. Por eso el robot usa sólo la parte baja del rango.

### Freno y rueda libre

Con las dos entradas de un canal al mismo nivel, el puente **frena** (cortocircuita el motor). El firmware frena 200 ms cuando salta el watchdog y después suelta a rueda libre, para no calentar el L298.

# Los motores: dos unidades que no son iguales

## Lo que se midió (2026-08-27)

Ventanas de 10 s a duty fijo, contando vueltas:

- La rueda **izquierda** da **2,11 veces** más velocidad por cuenta de duty que la derecha.
- Y a la vez **cuesta más arrancarla**.

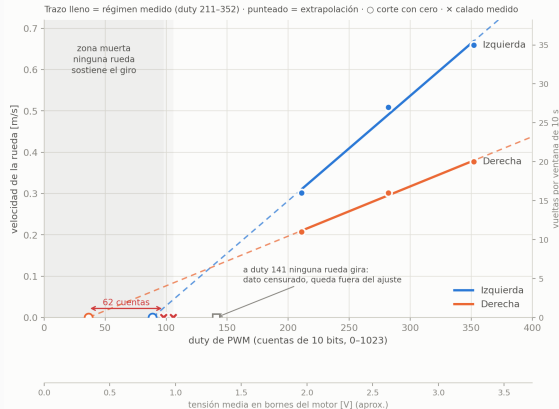
## La explicación

Las dos cosas juntas no son fricción: son una **reducción menor** (relación  $\sim 2,1:1$  entre las dos cajas). Sólo la derecha coincide con la chapa de “350 rpm”.

## Lo que eso obliga

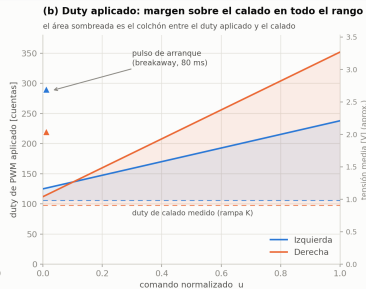
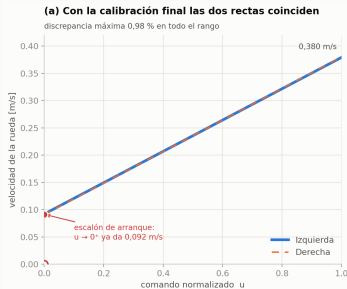
Cada rueda tiene su **propia recta** de duty a velocidad, con piso y techo propios. Un solo  $v_{\min}$  y un solo  $v_{\max}$  no alcanzan.

## Velocidad medida y recta ajustada, por rueda



Velocidad medida contra duty, por rueda. Azul: izquierda; naranja: derecha  
(cuaderno 01\_calibracion\_motores.ipynb).

# La calibración: una recta por rueda, con piso y techo



## El mapa (en el ESP32)

Para un comando  $u$  entre 0 y 1 de cada rueda:

$$\text{duty} = \text{PWM\_MIN} + (\text{PWM\_TOP} - \text{PWM\_MIN}) \cdot u$$

y  $\text{duty} = 0$  si  $u = 0$ . Con eso las dos ruedas siguen **la misma recta de velocidad**, con menos de 1 % de diferencia.

## La consecuencia que más pesa

**No hay arranque suave.** Cualquier  $u > 0$  da como mínimo 0,091 m/s; un comando de 1 no hace ir “despacito”. Para parar hay que mandar exactamente 0. De acá salen la zona muerta del control y varios de los problemas del piso.

| Constante del firmware  | Banco (27/08) | Piso (30/09) | Qué es                                                      |
|-------------------------|---------------|--------------|-------------------------------------------------------------|
| PWM_MIN izq / der       | 125 / 112     | 300 / 230    | Piso de la recta: el duty de la mínima velocidad sostenible |
| PWM_TOP izq / der       | 238 / 352     | 413 / 470    | Techo de la recta: el duty de $u = 1$                       |
| PWM_BREAKAWAY izq / der | 290 / 220     | 380 / 280    | Pulso de arranque, para vencer la fricción estática         |
| BREAKAWAY_MS            | 80 ms         | 150 ms       | Cuánto dura ese pulso                                       |

## Por qué hubo que rehacerla

La de banco se hizo **con las ruedas al aire**. En el piso la rueda izquierda —la de menos par, y encima la de menor duty— se **calaba con cualquier comando**: los motores pitaban y el robot no giraba.

Las constantes PWM\_\* de src/config.py son una copia informativa: las que valen están en el firmware. Se pusieron al día con los valores de piso el 1 de octubre (en la PC; falta copiarlas al Pi).

## El modelo que funcionó

La carga **suma un escalón de duty** (el par es corriente) y **no cambia la pendiente** (la velocidad es fuerza contraelectromotriz). Piso y techo suben lo mismo por rueda. Verificado: avance derecho, y 57 cm donde el modelo predecía 57.

## Lo que quedó

**Webcam USB Microsoft LifeCam Studio** en `/dev/video0`. Captura a  $640 \times 480$  en MJPG; la red trabaja a 320 px.

- Campo visual horizontal:  $\sim 52^\circ$  (estimado en el piso; se había supuesto 60).
- Alcance con el oso A4:  $\sim 1,3$  m.
- Montada al frente y muy baja (del orden de 10 cm del piso; no se midió). Ve los objetos desde abajo.

## Lo que la cámara mide

No mide distancia: mide **tamaño aparente**. Por eso el objeto con el que se calibra es parte del número.

## Lo que no funcionó: la Arducam

La cámara prevista era una Arducam UC-609 (sensor IMX219, cable de 15 pines) por el conector CSI. El Pi 5 **nunca la vio**: nada en `dmesg`, nada en el bus I2C de los conectores, con las cuatro orientaciones del cable. Como ese sensor sí está en la autodetección, la falla era eléctrica. Se descartó **por tiempo, no por diseño**.

## Dos cosas raras de la imagen

- Girando de corrido sale **movida**: YOLO no ve nada.
- A ratos llega **en franjas**, aun con los motores parados. Causa sin aislar.

## Sistema

- Raspberry Pi OS (Trixie), 64 bits, Python 3.13.
- Sin monitor ni teclado: se entra por SSH con la clave `id_ed25519_pi`. Nombre `pi-MB`, usuario `admin`.
- Todo el proyecto vive en `~/TPF/: src/`, `scripts_pi/`, `logs/` y el entorno `.venv/`.

## Entorno de Python

`torch 2.14`, `torchvision 0.29`, `ultralytics 8.4`, `opencv 5.0` y `pyserial`; todo instalado por rueda, nada compilado. El entorno pesa 5,6 GB, de los cuales 3,3 GB son librerías de CUDA que no se usan (pendiente de higiene).

## Térmico

Con YOLO real corriendo 150 s: 2400 MHz y sin recortes, con meseta en **73–75 °C**. A 80 °C empieza a recortar. (Un test sintético de cuatro bucles vacíos sí lo hacía recortar: es más severo que la inferencia.)

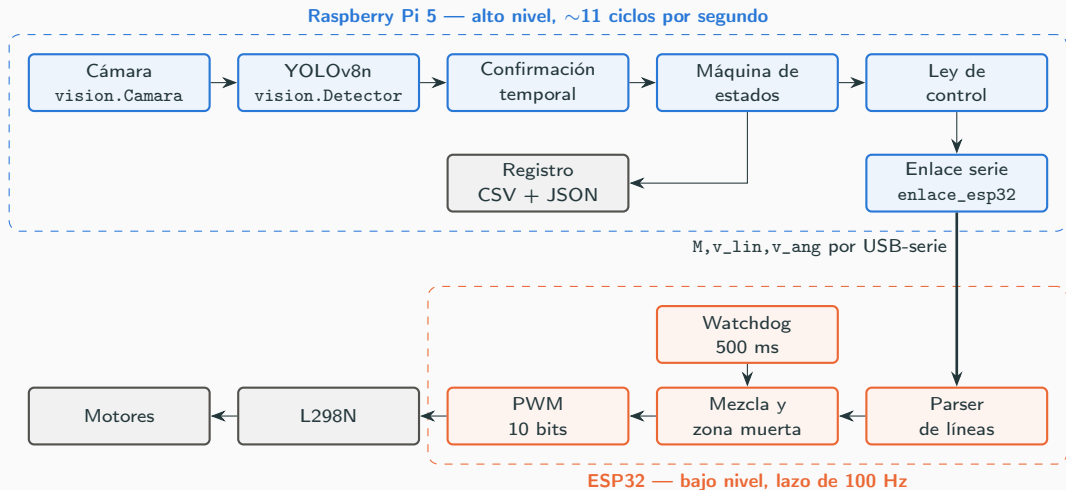
## Cómo se le pregunta al Pi cómo está

- `vcgencmd get_throttled`: 0x0 es todo bien. El bit 0 es bajo voltaje ahora; el 16, que lo hubo desde el arranque.
- `vcgencmd pmic_read_adc EXT5V_V`: la tensión de entrada (4,9 a 5,1 V).
- `vcgencmd measure_temp`.

## Arquitectura

---

## Dos niveles, con una frontera clara



**El principio:** el Pi nunca toca un PWM y el ESP32 nunca decide un comportamiento. Cada nivel se depura por separado.

# Qué sabe cada nivel, y qué ignora a propósito

## Raspberry Pi: la intención

- Sabe qué hay en la imagen y qué conviene hacer.
- Habla en **intención normalizada**: avanzar y girar entre  $-100$  y  $100$ .
- **Ignora** cuántas cuentas de PWM hacen falta, que las ruedas son distintas y que existe un puente H.
- No es de tiempo real: un ciclo tarda  $\sim 90$  ms y a veces más.

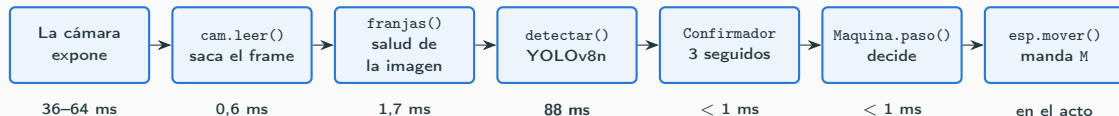
## ESP32: el hardware

- Sabe traducir intención a duty, rueda por rueda, con la calibración medida.
- Garantiza que el robot **frena solo** si se queda sin órdenes.
- **Ignora** qué es un oso o una mochila, y en qué estado está el comportamiento.
- Sí es de tiempo real: su lazo corre cada 10 ms.

## Por qué la zona muerta se compensa en el ESP32

Los pisos y techos de cada rueda están en **cuentas de PWM**, que es justo lo que el protocolo abstrae. Si se recalibra o se cambia un motor, se toca el firmware y **el Pi no se entera**. Fue lo que pasó el 30 de septiembre: se recalibró en el piso sin cambiar una línea de Python.

# La vida de un frame: de la foto a las ruedas



## Lo que cuesta

Casi todo el ciclo es **la red**: 88 de ~90 ms. La máquina de estados y el enlace cuestan alrededor del 1%. Por eso el tamaño de inferencia es la única palanca real del FPS.

## Después del comando

registro.fila() escribe una línea del CSV y, si se pidió --grabar, el frame anotado va al video.

## La latencia que no se ve

Sacar el frame de la cola tarda 0,6 ms, pero ese frame **ya tiene 36 a 64 ms de edad**. De foto a decisión pasan ~150 ms; con el retardo de los motores, ~0,25 s. A 0,25 m/s son unos 6 cm “a ciegas”.

## El orden importa

La salud de la imagen se mide **antes** de dibujar las cajas encima.

## Tres relojes distintos, desacoplados

| Quién                               | Cadencia          | Qué hace a ese ritmo                                                                                                                     |
|-------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Lazo de visión (Pi)                 | ~11 Hz, irregular | Un ciclo por frame: detectar, decidir, fijar el comando. Si un frame tarda, el ciclo tarda.                                              |
| Latido del enlace (Pi, hilo aparte) | 10 Hz, regular    | Repite el último comando al ESP32 <b>mientras siga fresco</b> . Si el comando <i>cambia</i> , se manda en el acto sin esperar al latido. |
| Lector del enlace (Pi, hilo aparte) | lo que llegue     | Lee la telemetría D, ... que el ESP32 manda a 20 Hz.                                                                                     |
| Lazo de control (ESP32)             | 100 Hz            | Revisa el watchdog y aplica el duty de cada rueda.                                                                                       |
| Telemetría (ESP32)                  | 20 Hz             | Manda D,dist,enc_izq,enc_der. Hoy siempre D,-1,0,0.                                                                                      |

### Por qué hay un latido

El ESP32 frena si pasa 500 ms sin un comando. El lazo de visión es irregular: un frame lento haría frenar el robot en medio de una maniobra. El latido desacopla las dos cadencias.

### Y por qué el latido no puede ser ingenuo

Repetir el último comando *para siempre* anularía el watchdog: si el lazo se cuelga, el hilo del latido seguiría diciendo "andá". Por eso sólo repite comandos con menos de 0,40 s de edad (VIGENCIA\_COMANDO\_S).

## Regla 1 — Todos los parámetros viven en config.py

Nada de constantes sueltas en el código. Cada valor lleva al lado **de dónde salió**: medido (con fecha), calculado o pendiente. Los días de piso se tocan muchas veces, y un número sin su contexto se usa mal.

## Regla 2 — Las capas que deciden no tocan hardware

**Deciden** (funciones puras, reciben el tiempo por argumento):

- control.py
- estados.py
- vision.Confirmador

**Hablan con el mundo:**

- vision.Camara, vision.Detector
- enlace\_esp32.py
- main.py (el cableado)

## Lo que eso compra

El comportamiento entero se verifica **en la PC**, sin cámara, sin ESP32 y sin robot (prueba\_estados.py, 79 verificaciones), y se puede correr contra un robot simulado (simular\_piso.py). Así se encontró el bucle de huidas antes de que hubiera ruedas girando.

## Qué pasa cuando algo falla

| Falla                                                     | Quién reacciona                                                   | Resultado                                |
|-----------------------------------------------------------|-------------------------------------------------------------------|------------------------------------------|
| El lazo de visión se cuelga (el programa sigue vivo)      | El latido ve que el comando tiene más de 0,40 s y manda M,0,0     | Frena en menos de medio segundo          |
| El programa muere, se corta el SSH o se desconecta el USB | El watchdog del ESP32: 500 ms sin M                               | Freno activo 200 ms, después rueda libre |
| La cámara no entrega un frame                             | main.py corta el lazo y sale por el camino normal                 | Frena al salir                           |
| Ctrl+C                                                    | Baja una bandera; el lazo termina su ciclo y sale del with Enlace | Frena, cierra el CSV y el video          |
| Una detección aislada falsa                               | El Confirmador pide 3 frames seguidos                             | A lo sumo una pausa de un frame          |
| La imagen llega rota                                      | Nadie reacciona: <b>sólo se registra</b> (columna franjas)        | El robot queda "ciego" y sigue buscando  |
| Un obstáculo en el camino                                 | <b>Nadie:</b> no hay ultrasónico                                  | Choca. Área despejada y alguien cerca    |

El corte que siempre funciona es físico: **sacar el conector de 12 V** de la UPS.

## El código del Pi, archivo por archivo

---

TPF/

|                               |                       |
|-------------------------------|-----------------------|
| PLAN.md                       | plan y bitacora       |
| HARDWARE.md                   | analisis electrico    |
| RUNBOOK.md                    | paso a paso           |
| GUIA_PRESENTACION.md (+ .pdf) |                       |
| firmware/                     |                       |
| esp32_control/                | firmware definitivo   |
| esp32_motor_test/             | prueba de cableado    |
| src/                          | el codigo del Pi      |
| scripts_pi/                   | puesta en marcha      |
| piso/                         | pruebas en el piso    |
| scripts_auxiliares/           | cuadernos             |
| presentacion/                 | arquitectura (agosto) |
| presentacion_resumen/         | este documento        |
| logs/                         | registros de corridas |
| media/                        | video y evidencia     |

## Los cuatro documentos

- PLAN.md: el estado actual (§1), la especificación (§6 a §8) y la **bitácora** de cada jornada (§13). Es el que manda.
- HARDWARE.md: motores, L298, alimentación, cableado; cada medición con su fecha.
- RUNBOOK.md: los pasos numerados para repetir cada cosa.
- GUIA\_PRESENTACION.md: cómo operar el robot sin ayuda el día de la presentación.

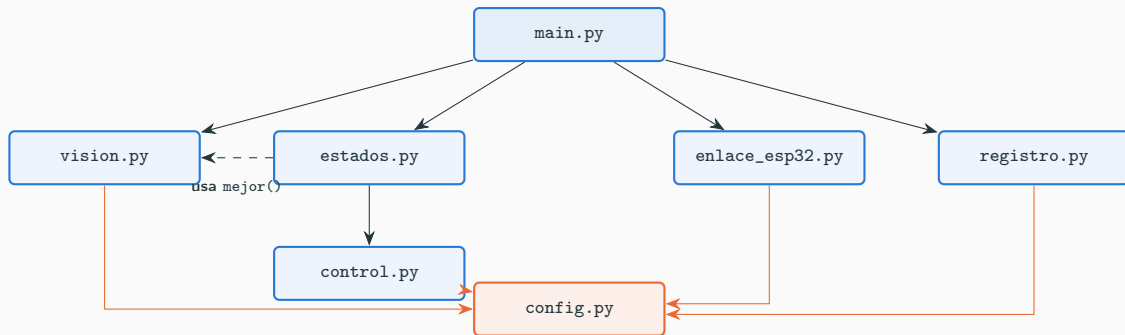
## Dónde corre cada cosa

src/ y scripts\_pi/ se copian al Pi, en ~/TPF/. El firmware se compila en la PC con PlatformIO. Los cuadernos y este documento viven sólo en la PC.

| Archivo                                     | Líneas | Qué es                                                           |
|---------------------------------------------|--------|------------------------------------------------------------------|
| <b>El robot</b> — lo que corre en cada demo |        |                                                                  |
| config.py                                   | 484    | Todos los parámetros, cada uno con su origen                     |
| vision.py                                   | 386    | Cámara, detector YOLO, confirmación temporal, salud de la imagen |
| control.py                                  | 178    | La ley de control: de una caja a (v_lin, v_ang)                  |
| estados.py                                  | 349    | La máquina de estados: el comportamiento                         |
| enlace_esp32.py                             | 366    | El protocolo serie con el ESP32, el latido y la parada segura    |
| main.py                                     | 245    | El lazo principal: sólo cablea los anteriores                    |
| registro.py                                 | 136    | El CSV y el JSON de cada corrida, y el video                     |
| <b>Las herramientas de medición</b>         |        |                                                                  |
| bench_vision.py                             | 423    | Mide el FPS real; modo “en vivo” para ver qué detecta            |
| analizar_corrida.py                         | 305    | Saca las métricas de un CSV de corrida                           |
| <b>La verificación sin hardware</b>         |        |                                                                  |
| prueba_estados.py                           | 605    | 79 verificaciones del comportamiento, con el tiempo simulado     |
| simular_piso.py                             | 318    | La máquina real contra un robot cinemático simulado              |

Más requirements.txt y los pesos del modelo, yolov8n.pt. Los nombres están en español, como el resto del proyecto.

## Quién usa a quién



### Cómo leerlo

`main.py` es el único que conoce a todos. Las flechas naranjas son "lee sus parámetros de `config.py`": todos lo hacen.

### Lo que *no* hay

`estados.py` y `control.py` no importan ni la cámara ni el puerto serie. Por eso las herramientas de verificación los usan directamente, sin robot.

### Qué es

Un módulo de constantes, sin lógica (salvo dos funciones de conversión). Cada valor tiene al lado un comentario que dice **de dónde salió**: medido (con fecha), calculado, o pendiente. Más de la mitad del archivo son esos comentarios.

### Sus secciones

0. Rutas (DIR\_LOGS, DIR\_MEDIA).
1. Cámara.
2. Detección (modelo, clases, umbrales, confirmación).
3. Ley de control y BUSCAR.
4. Tracción (lo que hace el firmware, replicado).
5. Estados: HUIR y ultrasónico.
6. Enlace serie.
7. Métricas.

### Las dos funciones

- `velocidad_real(u)`: de un comando  $u$  entre 0 y 1 a m/s, con el escalón de arranque:  
 $v = 0,0915 + 0,289 u$  para  $u > 0$ .
- `comando_para_velocidad(v)`: la inversa.

### Las marcas de los comentarios

Tilde verde: medido. Cuadrado vacío: puesto a ojo, falta medir. Círculo rojo: una trampa que ya costó una jornada.

### Cómo se cambia un parámetro para una sola corrida

`corrida_piso.sh etiqueta 25 "nota" KP_ANG=0.4`: lo cambia, corre, y lo restaura al terminar.

### Deteccion (un dato inmutable)

Una caja ya convertida a lo que el control necesita: clase, conf, la caja en píxeles (x, y, w, h) y los dos números de la ley de control:

- `error_x`: de  $-1$  (borde izquierdo) a  $+1$  (derecho).
- `area_ratio`: fracción de la imagen que ocupa la caja.

### Detector

Envuelve a YOLOv8n. Al construirse resuelve los **nombres** de las clases contra `model.names`; si el modelo no las conoce, falla ahí. `detectar(frame)` devuelve la lista de `Deteccion` y los tiempos de inferencia y posproceso.

### Dos cosas que agregó el piso

- **Alias** (ALIAS\_CLASES): lo que el modelo llama suitcase sale del detector ya renombrado a backpack. El resto del sistema no se entera.
- **Umbral por clase** (UMBRAL\_POR\_CLASE): a YOLO se le pide con el umbral más bajo y cada caja se filtra con el de su clase (0,20 la mochila, 0,35 el oso).

### Otros detalles

- `calentar()`: tres inferencias de descarte. La primera tarda varias veces lo normal.
- El filtrado por clase lo hace la propia librería (`classes=...`): no se procesan las otras 78 clases.

### Confirmador

Lleva, por clase, cuántos frames seguidos se la vio y cuántos no. `actualizar(clases)` devuelve el conjunto de clases **confirmadas**: entra con 3 frames seguidos (`N_CONFIRMAR`), sale con 5 sin verla (`M_PERDER`).

### `mejor(detecciones, clase)`

La caja más confiable de una clase, o `None`. Es lo único de este módulo que usa la máquina de estados.

### `dibujar()`

Anota el frame con las cajas y el estado. Sólo para el video y para depurar.

### Camara

Abstrae las dos cámaras posibles: CSI (Picamera2) o USB (OpenCV sobre V4L2). Hoy siempre USB. Fija **primero el formato** (MJPG) y después el tamaño, y pide un búfer de un solo frame para no trabajar sobre imágenes viejas.

### `franjas(frame)`

Fracción de las filas que son una **franja gris uniforme**: lo que deja un paquete USB perdido en un frame MJPG. 0,0 es un frame sano; arriba de 0,10 la detección ya no es confiable. Cuesta 1,7 ms y va a cada fila del registro.

### Qué es

Convierte una detección en un comando. Nada más: no toca la cámara, no toca el puerto y no recuerda nada entre llamadas.

### Las funciones

- `girar_hacia(error_x)` → `v_ang`, con zona muerta.
- `avanzar_hacia(area_ratio)` → `v_lin`, nunca negativo.
- `perseguir(deteccion)` → las dos, con el giro **acotado** a 0,7 veces el avance.
- `a_protocolo()`: de normalizado a entero entre `-100` y `100`, con tope.

### Las de diagnóstico

- `techo_efectivo_lineal()`: cuál es el `v_lin` máximo que la ley puede pedir, y **cuál de los dos límites manda** (la ganancia o `V_LIN_MAX`).
- `kp_lin_para_saturar()`: la ganancia mínima para que mande `V_LIN_MAX`.
- `describir()`: el resumen que `main.py` imprime al arrancar.

### Se puede correr solo

`python control.py` imprime una tabla de `error_x` a `v_ang` y de `area_ratio` a `v_lin` con los parámetros vigentes. Sirve para ver qué va a pedir la ley antes de soltar el robot.

### Comando

Lo que la máquina pide en un ciclo: `v_lin`, `v_ang`, `buzzer` (casi siempre `None`), `estado` y un motivo en texto. El motivo es lo que aparece en la consola y en el CSV: “barriendo”, “posible objetivo, freno a confirmar”...

### `Maquina.paso(...)`

Un ciclo de decisión. Recibe las clases confirmadas, las detecciones crudas del frame, la telemetría del ESP32 y **la hora**; devuelve un `Comando`. No lee ningún reloj por su cuenta.

### Lo que la máquina recuerda

- `estado` y `entrado_en`: en qué estado está y desde cuándo.
- `fase_huir`: 1, 2 o 3.
- `lado_amenaza`: de qué lado estaba la mochila.
- `huida_termino_en`: para el período sordo.
- `sentido_barrido`: hacia dónde gira al buscar.

### Un método por estado

`_buscar`, `_aproximar`, `_huir`, `_encontrado`. El detalle de cada uno está en la sección “La máquina de estados”.

### Enlace

Abre el puerto, espera 2 s a que el ESP32 termine de reiniciarse y lanza **dos hilos**:

- *lector*: lee líneas, descarta las que empiezan con # y guarda la última telemetría.
- *latido*: cada 0,1 s manda el comando vigente, o M,0,0 si ya venció.

### Lo que se le pide

- `mover(v_lin, v_ang)`: fija el comando. Si cambió, lo manda **en el acto**.
- `parar()`: M,0,0 ya.
- `buzzer(patron)`: un evento, se manda una vez.
- `estado`: la última telemetría.

### EstadoESP32

La última línea D interpretada. `dist_cm` es None cuando el ESP32 manda -1 ("sin lectura"); `obstaculo()` trata el "no sé" como "no". Hoy siempre es "no sé".

### Para probar sin ESP32

`Enlace(simular=True)` usa un puerto de mentira que contesta D,-1,0,0 y guarda lo que se le escribe. Es lo que usa `main.py --simular`.

### Sale siempre frenando

Se usa con `with`: al salir manda M,0,0, aunque haya habido una excepción.

```

cam, det = Camara(), Detector()
conf, maq = Confirmador(), Maquina()
with Enlace() as esp:
    det.calentar(cam.leer())
    esperar(--espera)          # soltar el robot
    reg = Registro(...)
    while seguir:
        frame = cam.leer()
        rotas = franjas(frame)
        dets = det.detectar(frame)
        confirmadas = conf.actualizar(dets)
        cmd = maq.paso(confirmadas, dets,
                        esp.estado, ahora)
        esp.mover(cmd.v_lin, cmd.v_ang)
        if cmd.buzzer: esp.buzzer(cmd.buzzer)
        reg.fila(..., rotas)
        if tiempo >= --duracion: break
# al salir del with: M,0,0

```

| Opción         | Qué hace                                               |
|----------------|--------------------------------------------------------|
| --duracion N   | Para solo a los N segundos.<br><b>Siempre ponerla.</b> |
| --espera N     | Segundos antes de arrancar                             |
| --simular      | No habla con el ESP32                                  |
| --silencioso   | Una línea cada 10 ciclos                               |
| --nota "..."   | Qué escenario es; va al JSON                           |
| --grabar       | Video anotado (baja ~1 FPS)                            |
| --sin-registro | No escribe el CSV                                      |
| --tam N        | Otro tamaño de inferencia                              |
| --puerto P     | Otro puerto serie                                      |
| --preview      | Ventana (no sirve por SSH)                             |

Ctrl+C no mata el proceso: baja una bandera y el lazo sale por su camino normal, que es el que frena.

| Columna del CSV      | Qué guarda                                    |
|----------------------|-----------------------------------------------|
| t_s, dt_ms           | Tiempo desde el arranque y duración del ciclo |
| estado, fase_huir    | Estado de la máquina                          |
| v_lin, v_ang, buzzer | El comando enviado                            |
| confirmadas          | Clases confirmadas, separadas por barra       |
| n_det                | Cuántas cajas hubo en el frame                |
| franjas              | Salud de la imagen (0 es sana)                |
| obj_conf/ex/ar/w/h   | La mejor caja <b>cruda</b> del oso            |
| amz_conf/ex/ar       | La mejor caja cruda de la mochila             |
| dist_cm              | Ultrasónico (vacío: sin lectura)              |
| motivo               | Por qué se mandó ese comando                  |

## Dos archivos por corrida

corrida\_AAAAMMDD\_HHMMSS.csv con una fila por ciclo, y un .json con **los parámetros de esa corrida** y la nota. Un CSV sin sus parámetros es un número sin contexto.

## Y el video

Con --grabar, un .avi con el frame anotado de cada ciclo: el frame  $k$  del video es la fila  $k$  del CSV.

## Por qué la caja cruda

Guardar la caja confirmada *o no* permite medir cuánto tardó la confirmación desde el primer avistamiento.

### Qué informa de una corrida

- **Lazo:** FPS, y duración del ciclo (mediana, percentil 95 y máximo).
- **Imagen:** frames con franjas; avisa “cámara a ciegas” si más del 20 % están muy dañados.
- **Estados:** tiempo en cada uno y cada transición.
- **Buscar:** cuánto tardó en confirmar desde que lo vio, y si el oso seguía en la imagen medio segundo después.
- **Aproximar:** error lateral (RMS y máximo), cruces de lado y pérdidas del objetivo.
- **Encontrado:** con qué área frenó y con cuál quedó.
- **Huir:** latencia de reacción y frames con mochila.

### Con varias corridas

`python analizar_corrida.py logs/corrida_*.csv`  
arma una tabla con una línea por corrida y el resumen: cuántas llegaron a ENCONTRADO y el tiempo medio. Es la herramienta para las 10 corridas por escenario.

### Lo que el CSV no puede decir

La distancia **real** a la que frenó, si chocó, y cuánto giró de verdad. El registro sabe qué pidió el Pi y qué vio la cámara; no sabe dónde quedó el robot. Eso se anota a mano, con cinta métrica.

### Dependencias

Sólo la biblioteca estándar: corre igual en el Pi y en la PC.

---

### Qué verifica cada grupo

---

- |   |                                                             |
|---|-------------------------------------------------------------|
| 1 | BUSCAR: barre, se reubica, alterna mirar y girar            |
| 2 | APROXIMAR: frena al acercarse, gira al lado correcto, llega |
| 3 | Pérdida del objetivo, y que lo busca por donde lo vio       |
| 4 | HUIR tiene prioridad sobre todo                             |
| 5 | Las tres fases de HUIR, en orden y sin reevaluar            |
| 6 | El período sordo después de huir                            |
| 7 | El ultrasónico: obstáculo propio contra ajeno               |
| 8 | Frenar el barrido al primer avistamiento                    |
| 9 | Todo lo que sale entra en el protocolo                      |
- 

### Cómo funciona

Una clase Escenario corre la máquina con un **reloj simulado** y detecciones inventadas, y junta los comandos que fueron saliendo. Corre en un segundo, en cualquier máquina.

### Para qué sirvió

Encontró, antes de que hubiera ruedas girando, que las huidas se reencadenaban solas. Y `demo.sh` chequeo la corre antes de cada demo.

### Lo que no garantiza

Que la decisión verificada sea *buena*. Una de estas verificaciones exigía, hasta el 1 de octubre, justo lo que hacía que el robot no huyera.

### simular\_piso.py: predecir antes de soltar el robot

Corre la **máquina y el confirmador reales** contra un robot diferencial simulado. Lo que se simula es el mundo; lo que decide es el mismo código del Pi.

- **Medido:** calibración de motores, FPS, edad del frame, alcance.
- **Estimado:** trocha, campo visual.
- **Supuesto:** inercia, probabilidad de detección.

Predijo que el barrido de corrido iba a fallar si la imagen se movía. Sirve para comparar variantes, no para creerle los números absolutos.

### bench\_vision.py: cuánto rinde la visión

Mide el FPS real de cámara más red, barriendo tamaños de inferencia y separando captura, inferencia y posproceso.

- Sin argumentos: el barrido, con resultados en logs/.
- `--en-vivo`: imprime lo que detecta frame a frame y guarda la foto anotada en logs/vivo.jpg. Es lo que usa `demo.sh` ver.
- Tiene una cámara sintética para medir sin cámara conectada.

De acá salió `TAM_INFERENCIA = 320`.

## El firmware del ESP32

---

## Dónde está

`firmware/esp32_control/src/main.cpp`: un solo archivo de unas 1360 líneas, en C con el marco de Arduino, compilado con PlatformIO. La carpeta `firmware/` es un repositorio git aparte.

## Qué hace

- Recibe intención de movimiento por el puerto serie.
- La traduce a duty de PWM, rueda por rueda.
- Frena solo si se queda sin órdenes.
- Devuelve telemetría 20 veces por segundo.
- Ofrece un modo manual y un modo de calibración para trabajar sin el Pi.

## Qué no hace

**No decide nada.** No sabe qué es un oso, ni en qué estado está el comportamiento. Si el Pi le pide algo absurdo, lo ejecuta (acotado a  $\pm 100$ ).

## El otro firmware

`esp32_motor_test/` fue el de la prueba de cableado de julio: mover los motores con teclas. Quedó como registro; no se usa.

## Cómo saber qué firmware tiene cargado

`demo.sh` `firmware` le pide sus parámetros. Tiene que decir `v_min izq/der: 300 / 230`. Si dice `125 / 112` es el de banco, con el que la rueda izquierda no se mueve apoyada.

```
loop():  
    serial_poll()      # lee lineas, las procesa  
    ping_poll()        # recoge el eco (ultrasonico)  
    buzz_update()      # avanza el patron del buzzer  
    ramp_update()      # calibracion: rampa K  
    window_update()    # calibracion: ventana V  
  
    cada 60 ms: ping_send()  
    cada 10 ms:          # 100 Hz  
        watchdog_update()  
        si esta en calibracion:  
            duty crudo, sin compensar  
        si no esta frenando el watchdog:  
            wheel_apply(izquierda)  
            wheel_apply(derecha)  
    cada 50 ms: telemetry_send()    # 20 Hz
```

## Nada bloquea

No hay ningún `delay()` en el lazo. Cada tarea mira el reloj y hace su parte si le toca. El ultrasónico se mide **por interrupción**: un `pulseIn()` con 30 ms de espera habría trabado un lazo de 10 ms.

## Qué pasa cuando llega una línea

`serial_poll()` junta caracteres hasta el salto de línea y llama a `process_line()`, que distingue:

- M, ... y B, ...: el protocolo.
- K, V, C: calibración.
- una letra sola: modo manual.

Un M guarda el comando de cada rueda y pone el watchdog en cero.

## De $M, v_{lin}, v_{ang}$ al duty de cada rueda



| Comando | Rueda izquierda | Rueda derecha   | Qué hace el robot                                                                |
|---------|-----------------|-----------------|----------------------------------------------------------------------------------|
| M,55,0  | +55 → duty 362  | +55 → duty 362  | Avanza derecho a 0,25 m/s. El mismo duty es casualidad: las rectas se cruzan ahí |
| M,0,35  | +35 → duty 339  | -35 → duty -314 | Gira a la derecha en el lugar. La izquierda arranca con 380 los primeros 150 ms  |
| M,13,6  | +19 → duty 321  | +7 → duty 246   | Avanza despacio doblando a la derecha                                            |
| M,0,0   | 0               | 0               | Para. Es el único comando que detiene las ruedas                                 |

### Si la mezcla se pasa de 100

Se escala el **par completo**, no cada rueda por separado: recortar una sola cambiaría la curvatura pedida.

### El signo

$v_{ang}$  positivo gira a la **derecha**, igual que  $error_x$  positivo es “objetivo a la derecha”. El número sale de la ley de control y entra al protocolo sin cambiar de signo.

| Constante             | Valor     | Qué hace                                                 |
|-----------------------|-----------|----------------------------------------------------------|
| PWM_RES_BITS, PWM_MAX | 10, 1023  | Resolución del PWM                                       |
| PWM_FREQ_HZ           | 1000      | Frecuencia del PWM de los motores (el pitido que se oye) |
| PWM_MIN_LEFT/RIGHT    | 300 / 230 | Piso de cada recta: el duty de $u \rightarrow 0^+$       |
| PWM_TOP_LEFT/RIGHT    | 413 / 470 | Techo de cada recta: el duty de $u = 1$                  |
| PWM_BREAKAWAY_*       | 380 / 280 | Duty mínimo durante el pulso de arranque                 |
| BREAKAWAY_MS          | 150       | Duración del pulso de arranque, al salir de parado       |
| CMD_TIMEOUT_MS        | 500       | Watchdog: sin comando M durante este tiempo, frena       |
| WD_BRAKE_MS           | 200       | Cuánto frena activo antes de soltar a rueda libre        |
| CONTROL_PERIOD_MS     | 10        | Período del lazo de control (100 Hz)                     |
| TELEMETRY_PERIOD_MS   | 50        | Período de la telemetría (20 Hz)                         |
| PIN_SONAR, PIN_BUZZER | 12, 14    | Los pines del ultrasónico y del buzzer                   |
| PING_PERIOD_MS        | 60        | Ultrasónico: tiempo entre disparos                       |
| ECHO_TIMEOUT_US       | 30000     | Ultrasónico: sin eco en este tiempo, lectura $-1$        |
| BUZZER_PASSIVE        | 0         | El buzzer es activo: suena solo con nivel alto           |
| BUZZ_ALARM_ON/OFF_MS  | 150 / 150 | Ritmo de la alarma: suena y calla                        |

Las seis primeras filas de motores son **la calibración**: cambiarlas es recompilar y volver a grabar el ESP32. El ultrasónico es un PING))) de **un solo pin** (dispara y escucha por el mismo cable): cómo lo maneja el firmware está en la sección “El 2 de octubre”.

## Watchdog

Si pasan **500 ms** sin un comando M:

1. pone los dos comandos en cero,
2. **frena activo** 200 ms (las dos entradas del puente al mismo nivel),
3. suelta a rueda libre, para no calentar el L298,
4. avisa por el puerto con una línea de diagnóstico.

Arranca ya “disparado”: el robot no se mueve hasta el primer comando. En modo manual está **desactivado**.

## Pulso de arranque (*breakaway*)

Arrancar una rueda parada cuesta más que mantenerla girando. Cuando el duty pasa de cero a algo, durante los primeros 150 ms el firmware aplica **al menos** 380 (izquierda) o 280 (derecha), y después baja al duty pedido.

## El costo de ese pulso

Cada vez que una rueda arranca o **cambia de sentido**, pega un empujón. En el barrido a pasos eso es buena parte de cada paso; al acercarse al oso, una rueda que se invertía sacudía al robot (ver “La ley de control”).

# Los tres modos del firmware

## Protocolo (el normal)

M,<v\_lin>,<v\_ang> y B,<patron>. Con watchdog y con compensación de zona muerta. Es lo que usa el Pi.

Cualquier M vuelve a este modo desde los otros.

## Manual (banco)

Una letra y Enter, desde un monitor serie:

- w s a d: mover.
- x: parar; b: frenar.
- 1 2: una rueda sola.
- + -: velocidad.
- z: zona muerta sí/no.
- e: ver parámetros.
- p: volver a protocolo.

**Sin watchdog.**

## Calibración

Duty **crudo**, sin compensar:

- K,r,duty0: rampa hacia abajo, para ver a qué duty se detiene la rueda.
- V,r,duty,ms: ventana cronometrada, para contar vueltas.
- C,r,duty: duty fijo.

r es 0 izquierda, 1 derecha, 2 las dos.

**Sin watchdog.**

## Para qué sirvió cada uno

El modo manual, para la prueba de recta y el primer cableado. La calibración, para medir las rectas de agosto (K y V) y los umbrales en el piso de septiembre (C, desde scripts\_pi/piso/umbral\_piso.py).

## Compilar (en la PC)

Con PlatformIO, desde `firmware/esp32_control/`: `pio run`. Deja los binarios en `.pio/build/esp32dev/`.

## Grabar desde el Pi, sin mover cables

El ESP32 vive enchufado al Pi, así que se copian los cuatro binarios al Pi y se graban con `esptool` por `/dev/ttyUSB0`. El procedimiento completo, con los desplazamientos de cada binario, está en `RUNBOOK.md`, sección “Flashear el ESP32 desde el Pi”.

## Antes de grabar: sacar los 12 V

Mientras se graba, los pines del ESP32 quedan flotando y el L298 puede dar un tirón a los motores.

## Después de grabar

`demo.sh firmware` para confirmar los parámetros cargados, y una prueba corta con `scripts_pi/piso/escalones.py` (comandos por el protocolo, con watchdog) antes de correr el comportamiento.

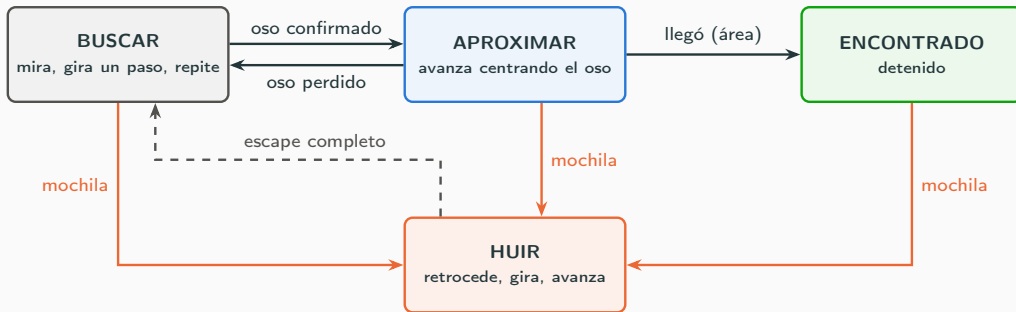
## Cuándo hubo que hacerlo

Una sola vez desde agosto: el 30 de septiembre, para cargar la calibración de piso.

## La máquina de estados

---

## Los cuatro estados y sus transiciones



### Lo que hay que mirar

Las flechas naranjas salen de **los tres** estados: la mochila interrumpe cualquier cosa. Y **ENCONTRADO** no tiene salida propia: el robot queda detenido hasta que termina la corrida o aparece la mochila.

### “Confirmado” no es “visto”

Las transiciones usan las clases **confirmadas** (3 frames seguidos). Lo “visto” en un solo frame no cambia el estado: a lo sumo hace que el robot se quede quieto para confirmar.

## 1. Huir tiene prioridad incondicional

Si la amenaza está confirmada, se huye: aunque el oso esté a la vista, aunque falten diez centímetros para llegar, aunque el robot ya esté en ENCONTRADO.

## 2. Una huida no se reevalúa a mitad de camino

Las fases 2 y 3 giran y avanzan **a ciegas**: la cámara pierde la mochila y eso es parte del diseño. Si se reevaluara, verla de nuevo reiniciaría la maniobra para siempre.

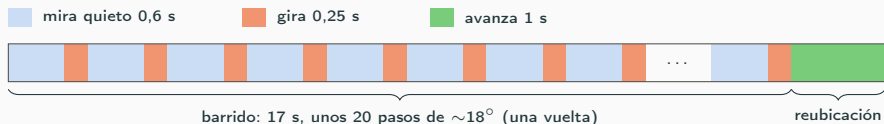
## 3. El buzzer es un evento, no un estado

La orden sale sólo en el ciclo de la transición que la provoca. Si no, el Pi mandaría una orden de buzzer por frame.

## 4. Después de huir hay un período sordo a la amenaza

No estaba en el diseño: lo encontraron las verificaciones. Al terminar de huir la mochila ya no está en cámara, pero el confirmador la sostiene 5 frames más, y sin los 2 s de sordera esa inercia dispara una segunda huida idéntica.

## BUSCAR: mirar, girar un paso, repetir



### Las tres partes

- **Mirando** (0, 0): quieto, con la imagen nítida. Unos 6 frames por pausa.
- **Barriendo** (0, 35): un pulso de giro de 0,25 s.
- **Reubicando** (30, 0): al terminar la vuelta, avanza  $\sim 18$  cm y empieza otra. Sin esto barrería para siempre la misma escena.

### Dos atajos

- **Frena al ver**: si en un frame aparece el oso o la mochila, aunque sin confirmar, **no gira**: se queda quieto para que los frames que confirman salgan nítidos.
- **Busca por donde lo vio**: si viene de perder el oso, gira hacia el lado donde estaba (`sentido_barrido`).

### Salidas

Oso confirmado  $\rightarrow$  APROXIMAR. Mochila confirmada  $\rightarrow$  HUIR.

| Cuándo           | Cómo era                          | Qué pasó en el piso                                                                                                                                          |
|------------------|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Diseño original  | Girar de corrido a velocidad baja | A $124^\circ/\text{s}$ la imagen sale <b>movida</b> : YOLO no reconoció el oso en ningún frame, aunque le pasó por delante varias veces.                     |
| 30 de septiembre | A pasos: girar 0,4 s, mirar 0,6 s | Cada paso renovaba el 61 % de la imagen ( $\sim 32^\circ$ ). El oso tiene que entrar entero: el margen era mínimo, y el barrido de 9 s no cerraba la vuelta. |
| 1 de octubre     | Pasos de 0,25 s; barrido de 17 s  | 35 % de la imagen por paso ( $\sim 18^\circ$ ): el oso queda entero en dos pausas seguidas. Encontró el oso a $90^\circ$ en 15 a 17 pasos.                   |
| 1 de octubre     | Primero mirar, después girar      | Con el orden inverso, lo primero que hacía el robot era apartar la vista de lo que tenía enfrente. Con la mochila adelante, no huía.                         |
| 1 de octubre     | Frenar al ver también la mochila  | La veía dos frames, giraba igual, el tercero salía movido y nunca la confirmaba.                                                                             |

### La medición que ordenó la discusión

`scripts_pi/piso/paso_giro.py` da pulsos de giro y mide con la propia cámara cuánto se corrió la escena entre una pausa y la siguiente. Lo que decide si el barrido tiene huecos es **qué fracción de la imagen se renueva por paso**, y eso no necesita saber el campo visual.

### Qué hace APROXIMAR en cada ciclo, en este orden

1. ¿El oso dejó de estar confirmado? → anota de qué lado se lo vio por última vez y vuelve a BUSCAR.
2. ¿Está confirmado pero no hay caja en este frame? → **frena** y espera el próximo. Seguir sería andar a ciegas.
3. ¿El área de la caja alcanzó AREA\_OBJETIVO, o el ultrasónico lee 20 cm o menos y lo de adelante es el oso? → ENCONTRADO, con el pitido de hallazgo.
4. ¿El ultrasónico ve algo a 20 cm o menos que **no** es el oso? → desvío, girando hacia el lado contrario.
5. Si no: control.perseguir() → avanzar y girar hacia el oso.

### ENCONTRADO

Manda 0, 0 en cada ciclo. Es terminal para la demo: no vuelve a buscar. Lo único que lo saca es la mochila.

### Dónde decide y dónde queda

AREA\_OBJETIVO es el tamaño con el que el robot **decide** frenar. Entre la decisión y la detención sigue avanzando (frame viejo más inercia). Con 0,35 decide a unos 38 cm y queda a  $\sim 30$ .

### Cuánto tarda

Desde 1 m: unos 2,5 s de aproximación. Desde que lo confirma de frente hasta frenar, 3 s.

## HUIR: tres fases con un solo reloj



| Fase         | Dura  | Comando       | Qué hace                                                                                                               |
|--------------|-------|---------------|------------------------------------------------------------------------------------------------------------------------|
| 1. Retroceso | 1,5 s | -40, $\mp 25$ | Marcha atrás $\sim 30$ cm, girando hacia el lado <b>opuesto</b> a la mochila. Va más lento que el avance: es a ciegas. |
| 2. Giro      | 1,2 s | 0, $\mp 60$   | Sigue girando <b>para el mismo lado</b> que la fase 1, hasta quedar de espaldas.                                       |
| 3. Avance    | 2,5 s | 60, 0         | Se aleja $\sim 65$ cm. Con algo a 30 cm o menos adelante, <b>gira en el lugar</b> en vez de avanzar (0, 60).           |
| Después      | 2,0 s | (en BUSCAR)   | Período sordo: ignora la mochila.                                                                                      |

### De qué lado gira

Al entrar a HUIR guarda el signo de `error_x` de la mochila (`lado_amenaza`). Si en ese frame no hay caja, usa el último lado conocido.

### Lo que sigue a ojo

El giro depende del lado: en marcha atrás el robot rota más rápido hacia la izquierda que hacia la derecha. Con 1,3 s se pasó de media vuelta; 1,2 s no se midió.

## El orden en que decide Maquina.paso()

```
paso(confirmadas, detecciones, esp, ahora):  
  
    si veo el oso en este frame:  
        recuerdo su error_x          # para buscarlo  
  
    si la mochila esta confirmada  
        y no estoy huyendo  
        y no estoy en el periodo sordo:  
            recuerdo de que lado esta  
            paso a HUIR (alarma)  
  
    segun el estado:  
        BUSCAR      -> _buscar()  
        APROXIMAR   -> _aproximar()  
        HUIR        -> _huir()  
        ENCONTRADO  -> detenido
```

### El arbitraje, en una tabla

| Situación                  | Resultado    |
|----------------------------|--------------|
| Oso y mochila a la vez     | Huye         |
| Mochila durante la huida   | La ignora    |
| Mochila, 2 s tras huir     | La ignora    |
| Mochila, robot detenido    | Huye         |
| Oso visto 1 o 2 frames     | Queda quieto |
| Mochila vista 1 o 2 frames | Queda quieto |

### Por qué la mochila va primero

La comprobación de la amenaza está **antes** del reparto por estado: ningún estado puede “olvidarse” de mirarla.

## El ultrasonido en la máquina de estados (desde el 2 de octubre)

| Dónde                   | Umbral | Qué hace                                                  | Motivo en pantalla                |
|-------------------------|--------|-----------------------------------------------------------|-----------------------------------|
| BUSCAR, pulso de avance | 30 cm  | Gira lo que dura el pulso, en vez de avanzar              | reubicando: obstáculo (...), giro |
| APROXIMAR, es el oso    | 20 cm  | Da por alcanzado el objetivo                              | llegué (sonar)                    |
| APROXIMAR, no es el oso | 20 cm  | Gira a tope hacia el lado contrario del oso               | obstáculo ajeno (...), desvío     |
| HUIR, fase 3            | 30 cm  | Gira en el lugar hasta tener el camino libre              | fase 3: obstáculo, esquivo        |
| HUIR, fases 1 y 2       | —      | Nada: el sensor mira adelante y el robot retrocede o gira | —                                 |

### Tres reglas

- **Dos umbrales:** 20 cm para llegar al oso (`DIST_PARADA_CM`), 30 para esquivar (`DIST_OBSTACULO_CM`).
- **Retención de 0,3 s:** una lectura “libre” suelta no alcanza para volver a avanzar.
- **Sin lectura no es distancia cero:** con el sensor desconectado, nada de esto actúa.

### ¿Es el oso lo que tengo adelante?

Sí, si la cámara lo ve con  $|error_x| < 0,7$  y ya ocupa el 60 % del área objetivo. En la práctica el robot frena antes por tamaño (con el sensor en  $\sim 28$  cm), así que el sonar queda de respaldo.

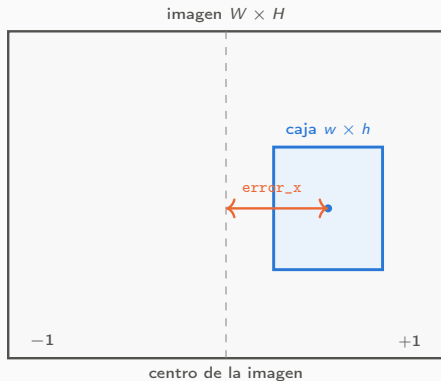
### De dónde salió cada número

De tres corridas del 2 de octubre, una por ajuste. Está contado en la sección “El 2 de octubre”.

## La ley de control

---

## De una caja en la imagen a dos números



### Error lateral: hacia dónde girar

$$error\_x = \frac{(x + w/2) - W/2}{W/2}$$

De  $-1$  (borde izquierdo) a  $+1$  (borde derecho). Cero es el centro. Con un campo visual de  $\sim 52^\circ$ , una unidad son unos  $26^\circ$ .

### Área relativa: cuánto falta

$$area\_ratio = \frac{w \cdot h}{W \cdot H}$$

La fracción de la imagen que ocupa la caja. Crece al acercarse, más o menos como  $1/d^2$ . No es distancia: es **tamaño aparente**.

Los dos números se calculan en `vision.py`, al armar cada `Deteccion`. La ley de control no vuelve a mirar píxeles.

## Giro

$$v_{ang} = \begin{cases} 0 & \text{si } |\text{error\_x}| < 0,08 \\ 100 \cdot K_{ang} \cdot \text{error\_x} & \text{si no} \end{cases}$$

acotado a  $\pm 60$  ( $V\_ANG\_MAX$ ).

## Avance

$$v_{lin} = 100 \cdot K_{lin} \cdot (A_{obj} - \text{area\_ratio})$$

acotado entre 0 y 55 ( $V\_LIN\_MAX$ ). Nunca negativo: la ley no retrocede.

Las dos son **proporcionales**: sin término integral ni derivativo. Los valores salen en unidades del protocolo (−100 a 100) y el ESP32 los convierte en duty.

## El recorte que acopla las dos

$$|v_{ang}| \leq 0,7 \cdot v_{lin}$$

Mientras avanza, la rueda de adentro nunca baja del 30 % del avance: ni se detiene ni se invierte.

## Con los valores de hoy

|                     |      |                     |
|---------------------|------|---------------------|
| KP_ANG              | 0,5  | ganancia de giro    |
| KP_LIN              | 2,4  | ganancia de avance  |
| AREA_OBJETIVO       | 0,35 | dónde decide frenar |
| ZONA_MUERTA_ERROR_X | 0,08 | casi centrado       |

## Qué pide la ley, con números

| área | distancia<br>aprox. | v_lin | velocidad | tope de<br>giro |
|------|---------------------|-------|-----------|-----------------|
| 0,03 | 1,3 m               | 55    | 0,25 m/s  | 38              |
| 0,05 | 1,0 m               | 55    | 0,25 m/s  | 38              |
| 0,12 | 65 cm               | 55    | 0,25 m/s  | 38              |
| 0,15 | 58 cm               | 48    | 0,23 m/s  | 33              |
| 0,20 | 50 cm               | 36    | 0,20 m/s  | 25              |
| 0,25 | 45 cm               | 24    | 0,16 m/s  | 16              |
| 0,30 | 41 cm               | 12    | 0,13 m/s  | 8               |
| 0,33 | 39 cm               | 5     | 0,11 m/s  | 3               |
| 0,35 | 38 cm               | 0     | frena     | —               |

Las distancias salen de  $\text{área} \approx 500/d^2$  (con  $d$  en cm), ajustado a los puntos medidos; valen  $\pm 10\%$ .

`python control.py` imprime estas tablas con los parámetros que estén cargados.

| error_x | v_ang               | nota                       |
|---------|---------------------|----------------------------|
| 0,05    | 0                   | zona muerta                |
| 0,10    | 5                   |                            |
| 0,20    | 10                  |                            |
| 0,40    | 20                  |                            |
| 0,60    | 30                  |                            |
| 0,80    | 40 $\rightarrow$ 38 | recortado a $0,7 \cdot 55$ |

### Cómo leerlo

Hasta los 65 cm va a velocidad máxima; de ahí en más baja en rampa hasta frenar. El tope de giro baja con la velocidad: cerca del oso corrige suave.

## Por qué cada pieza está ahí

### La zona muerta del giro

No es un ajuste fino. Como no hay arranque suave, con el oso casi centrado la ley pediría un giro “mínimo”, el robot lo ejecutaría a la velocidad mínima —que no es mínima— y oscilaría alrededor del centro.

### El techo de velocidad

$V\_LIN\_MAX$  existe porque los motores dan para 0,87 m/s y la visión no. A 0,25 m/s el robot avanza  $\sim 2,5$  cm por frame.

### Nunca marcha atrás

Si se pasa del área objetivo, el error se vuelve negativo y se recorta a cero. Alejarse es trabajo de HUIR.

### El recorte del giro: la pieza que costó tres intentos

1. **Sin recorte:** cerca del oso  $v\_lin$  es chico y un  $v\_ang$  moderado manda una rueda *para atrás*. Esa rueda sale con su pulso de arranque: sacudón. El oso saltó de  $-0,33$  a  $+0,21$  con el robot frenando.
2. **Recorte a  $v\_lin$ :** la rueda de adentro queda *detenida* y el robot pivota sobre ella a  $40-70^\circ/s$ . Terminó girado  $\sim 15^\circ$ .
3. **Recorte a  $0,5 \cdot v\_lin$ :** suave, pero corto justo al final. Perdió el oso por el borde.
4. **Recorte a  $0,7 \cdot v\_lin$ :** el vigente.

### La raíz común

Una rueda que cruza por cero no gira “un poquito menos”: pasa de 0,09 m/s a nada.

### La cuenta

La diferencia de comando entre ruedas es  $2 \cdot v_{ang}$ , que en velocidad son  $2 \cdot v_{ang}/100 \cdot 0,289$  m/s. Dividido por la trocha (0,18 m) da la velocidad de giro. Con  $v_{ang} = 100 \cdot K_{ang} \cdot error\_x$  y una unidad de error  $\approx 0,45$  rad:

$$\omega \approx 7,1 \cdot K_{ang} \cdot (\text{error en rad})$$

Eso es una **ganancia de lazo** en 1/s. Con un retardo  $\tau$  entre la foto y la respuesta de las ruedas, el lazo oscila cuando ganancia  $\times \tau$  se acerca a  $\pi/2 \approx 1,57$ .

| KP_ANG | ganancia | $\times 0,25$ s | en el piso                                                 |
|--------|----------|-----------------|------------------------------------------------------------|
| 0,8    | 5,7 1/s  | 1,4             | cruzó el centro y se fue al otro lado                      |
| 0,3    | 2,1 1/s  | 0,5             | centró bien por la derecha; por la izquierda perdió el oso |
| 0,5    | 3,5 1/s  | 0,9             | el vigente                                                 |

### Lo que la cuenta no tenía

- **Avanzando**, el giro realizado es más o menos la mitad del que predice la trocha medida girando en el lugar.
- El robot **se tuerce solo**:  $\sim 3,5^\circ/\text{s}$  a la izquierda acelerando y  $\sim 5^\circ/\text{s}$  a la derecha frenando.

Por eso 0,3 resultó flojo aunque “la cuenta daba”: la ganancia real era la mitad, y había una perturbación que vencer.

## El área no es distancia: qué mide `area_ratio`

| Distancia | área  | área · $d^2$ |
|-----------|-------|--------------|
| 30 cm     | 0,510 | 459          |
| 50 cm     | 0,222 | 555          |
| 70 cm     | 0,097 | 475          |
| 100 cm    | 0,050 | 500          |
| 110 cm    | 0,073 | 883          |

Medido con el oso de hoja A4, el 22 y el 30 de septiembre. Los cuatro primeros siguen el cuadrado inverso dentro de  $\pm 10\%$ ; el de 110 cm no (probable doble detección en el límite del alcance).

### El objeto es parte del número

Con el primer oso impreso, de 6,5 cm, el alcance era de  $\sim 45$  cm y frenar a 30 cm era `AREA_OBJETIVO` = 0.066. Con el de hoja A4 ( $\sim 20$  cm) el alcance pasó a más de 1 m y el mismo 0,066 habría sido frenar a 1 m. **Si cambia el objeto, se vuelve a medir.**

### Dónde decide y dónde queda

| Decide con         | Queda con | Distancia real |
|--------------------|-----------|----------------|
| 0,223 (obj. 0,222) | 0,333     | 37 cm (cinta)  |
| 0,375 (obj. 0,35)  | 0,573     | $\sim 28$ cm   |
| 0,366 (obj. 0,35)  | 0,49      | $\sim 30$ cm   |
| 0,419 (obj. 0,35)  | 0,662     | $\sim 26$ cm   |

Entre decidir y detenerse el área sigue creciendo: el frame tiene  $\sim 0,15$  s y el robot no frena en seco.

## Parámetro por parámetro

---

### Dónde están

Todos en `src/config.py`, en el orden de estas diapositivas. Los del firmware (duty de cada rueda, watchdog) están en la sección “El firmware del ESP32”.

### Qué dice cada fila

El nombre, el valor vigente al 1 de octubre, qué hace, y **de dónde salió**: si se midió, se calculó o se puso a ojo.

### Tres clases de número

- ■ **Medido**: hay una medición detrás, con fecha.
- ■ **Ajustado en el piso**: se llegó probando; anda, pero no hay una serie de corridas que lo respalde.
- □ **A ojo o sin uso**: valor inicial que nunca se revisó, o parámetro de algo que no está cableado.

### La advertencia de siempre

Cada número vale **en el contexto donde se midió**: este oso, esta mochila, este piso, esta cámara a esta altura. Es el hilo conductor de todo el trabajo.

| Parámetro                      | Valor    | Qué hace                                                            | De dónde sale                                                               |
|--------------------------------|----------|---------------------------------------------------------------------|-----------------------------------------------------------------------------|
| FUENTE_CAMARA                  | "usb"    | Qué cámara abrir: usb, csi o auto                                   | ■ Fijo en USB: la CSI nunca funcionó y así no se pierde tiempo intentándola |
| INDICE_CAMARA                  | 0        | Qué dispositivo: /dev/video0                                        | ■ Confirmado el 22/09                                                       |
| FORMATO_CAMARA                 | "MJPG"   | Formato en que la webcam manda la imagen por USB                    | ■ Obligatorio: con YUYV y los motores andando, 2,3 FPS e imagen en franjas  |
| ANCHO_CAPTURA,<br>ALTO_CAPTURA | 640, 480 | Resolución de captura (no es el tamaño de inferencia)               | <input type="checkbox"/> La nativa de la webcam; no se barrió               |
| FPS_CAMARA_SOLICITADO          | 30       | Lo que se le pide a la webcam                                       | <input type="checkbox"/> El lazo corre a ~11: sobran frames                 |
| FOV_H_GRAD                     | 52       | Campo visual horizontal, en grados. <b>Sólo lo usa el simulador</b> | ■ Estimado el 01/10: el oso reapareció tras ~4400 px de giro. Se suponía 60 |

**Captura contra inferencia:** la captura fija cuántos datos viajan por el USB; la inferencia, cuánto cuesta la red. Capturar a  $640 \times 480$  e inferir a 320 es reducir la imagen antes de dársela a YOLO.

| Parámetro        | Valor            | Qué hace                                                           | De dónde sale                                                             |
|------------------|------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------|
| PESOS_MODELO     | yolov8n.pt       | El archivo de pesos: YOLOv8 “nano”, 80 clases de COCO              | ■ El más chico de la familia; NCNN no hizo falta                          |
| TAM_INFERENCIA   | 320              | Lado de la imagen que ve la red. <b>La palanca del FPS</b>         | ■ Medido: 640 → 3,2 FPS, 480 → 5,2, <b>320</b> → <b>11,3</b> , 256 → 17,7 |
| UMBRAL_CONFIANZA | 0,35             | Confianza mínima para aceptar una caja (vale para el oso)          | □ Valor inicial; el oso da 0,85–0,95                                      |
| UMBRAL_POR_CLASE | mochila:<br>0,20 | Umbral propio de una clase, que pisa al general                    | ■ La mochila da 0,32–0,41 a 1,2 m. Sensible a objetos oscuros grandes     |
| UMBRAL_IOU       | 0,45             | Solapamiento para fundir cajas repetidas (supresión de no máximos) | □ El valor por defecto                                                    |

### El costo de TAM\_INFERENCIA

Bajar a 320 da FPS y **quita alcance**: a 480 o 640 se detectan objetos más chicos, o sea más lejanos. Se eligió mirando FPS y latencia, antes de haber medido el alcance. La decisión no cambió, pero el alcance se terminó arreglando agrandando el oso.

| Parámetro      | Valor                     | Qué hace                                                                       | De dónde sale                                    |
|----------------|---------------------------|--------------------------------------------------------------------------------|--------------------------------------------------|
| CLASE_OBJETIVO | teddy<br>bear             | La clase a la que se acerca                                                    | Decisión de diseño D3                            |
| CLASE_AMENAZA  | backpack                  | La clase de la que huye                                                        | Decisión de diseño D3                            |
| ALIAS_CLASES   | suitcase<br>→<br>backpack | Otros nombres del modelo para el <i>mismo</i> objeto; se renombran al detectar | ■ La mochila real es suitcase en 15 de 15 frames |
| N_CONFIRMAR    | 3                         | Frames seguidos para dar una clase por confirmada                              | □ Diseño (D6). A 10 FPS son $\sim 0,3$ s         |
| M_PERDER       | 5                         | Frames seguidos sin verla para darla por perdida                               | □ Diseño (D6). A 10 FPS son $\sim 0,5$ s         |

### Si N\_CONFIRMAR sube

Menos falsas maniobras, pero tarda más en reaccionar y pide que el robot esté quieto más tiempo en cada pausa del barrido.

### Si M\_PERDER sube

Aguanta más frames malos sin soltar el oso, pero la mochila queda "confirmada" más tiempo después de dejar de verla: el período sordo de HUIR tiene que ser más largo.

| Parámetro           | Valor | Qué hace                                                    | De dónde sale                                                |
|---------------------|-------|-------------------------------------------------------------|--------------------------------------------------------------|
| KP_ANG              | 0,5   | Ganancia del giro: cuánto $v_{ang}$ por unidad de $error_x$ | ■ 0,8 se pasaba de largo; 0,3 perdía el oso por la izquierda |
| ZONA_MUERTA_ERROR_X | 0,08  | Por debajo de este error no gira                            | □ Valor inicial. Evita oscilar con el oso casi centrado      |
| V_ANG_MAX           | 60    | Tope del giro. También es la velocidad de giro de HUIR      | ■ Da $165^\circ/s$ en régimen (30/09)                        |
| KP_LIN              | 2,4   | Ganancia del avance: fija dónde empieza la rampa de frenado | ■ Con este valor frena desde $\sim 65$ cm                    |
| AREA_OBJETIVO       | 0,35  | Área con la que <b>decide</b> frenar                        | ■ Decide a $\sim 38$ cm y queda a $\sim 30$                  |
| V_LIN_MAX           | 55    | Techo del avance: 0,25 m/s                                  | ■ Era 72 (0,30 m/s): no le daba tiempo a centrar             |

### Tres que se mueven juntos

La rampa de frenado empieza donde  $100 \cdot K_{lin} \cdot (A_{obj} - \text{área}) = V_{lin,max}$ . Cambiar cualquiera de los tres mueve ese punto. Y para que el techo *actúe* tiene que valer  $100 \cdot K_{lin} \cdot A_{obj} \geq V_{lin,max}$  (hoy 84 contra 55): al principio no se cumplía y V\_LIN\_MAX era un número decorativo.

| Parámetro            | Valor | Qué hace                                                            | De dónde sale                                                            |
|----------------------|-------|---------------------------------------------------------------------|--------------------------------------------------------------------------|
| V_ANG_BUSCAR         | 35    | Velocidad de giro de cada pulso del barrido                         | ■ Da 124°/s en régimen (30/09)                                           |
| BUSCAR_PASO_GIRO_S   | 0,25  | Duración de cada pulso de giro                                      | ■ Medido con la cámara: renueva el 35 % de la imagen ( $\sim 18^\circ$ ) |
| BUSCAR_PASO_MIRAR_S  | 0,6   | Cuánto se queda quieto mirando entre pulsos. Con 0, gira de corrido | ■ Unos 6 frames: alcanza para confirmar con 3                            |
| BUSCAR_GIRO_S        | 17    | Duración del barrido antes de reubicarse: una vuelta entera         | ■ 20 pasos de 0,85 s. Con 9 s no cerraba la vuelta                       |
| BUSCAR_AVANCE_S      | 1,0   | Duración del avance de reubicación                                  | □ Valor inicial                                                          |
| V_LIN_BUSCAR         | 30    | Velocidad de ese avance: 0,18 m/s, unos 18 cm                       | □ Valor inicial                                                          |
| BUSCAR_FRENAR_AL_VER | True  | Si ve el oso o la mochila sin confirmar, no gira                    | ■ Sin esto no confirmaba ninguno de los dos                              |

El **compromiso**: pasos más chicos no dejan huecos pero alargan la vuelta ( $\sim 17$  s hoy). La pausa de 0,6 s es la otra mitad de ese tiempo, y es la candidata a acortar.

| Parámetro           | Valor | Qué hace                                                      | De dónde sale                                                                                        |
|---------------------|-------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| HUIR_RETROCESO_S    | 1,5   | Duración de la fase 1 (marcha atrás)                          | <input type="checkbox"/> Valor inicial; el usuario lo dio por bueno                                  |
| HUIR_V_RETROCESO    | 40    | Velocidad de la marcha atrás: 0,21 m/s                        | <input type="checkbox"/> Más lenta que el avance: es a ciegas                                        |
| HUIR_GIRO_RETROCESO | 25    | Giro durante el retroceso, hacia el lado opuesto a la mochila | <input type="checkbox"/> Valor inicial. Aporta entre $\sim 10^\circ$ y $\sim 25^\circ$ según el lado |
| HUIR_GIRO_S         | 1,2   | Duración de la fase 2 (giro en el lugar a $V\_ANG\_MAX$ )     | <input checked="" type="checkbox"/> 1,1 daba $\sim 110^\circ$ ; 1,3 se pasó de media vuelta          |
| HUIR_AVANCE_S       | 2,5   | Duración de la fase 3 (alejarse)                              | <input type="checkbox"/> Valor inicial; $\sim 65$ cm, dado por bueno                                 |
| HUIR_V_AVANCE       | 60    | Velocidad de la fase 3: 0,27 m/s                              | <input type="checkbox"/> Valor inicial                                                               |
| HUIR_REFRACTARIO_S  | 2,0   | Período sordo a la mochila después de huir                    | <input checked="" type="checkbox"/> Tiene que superar $M\_PERDER/FPS \approx 0,5$ s                  |

**Por qué HUIR\_GIRO\_S no sale de una cuenta:** los  $165^\circ/s$  son de régimen; la fase 2 dura un segundo y arranca desde la marcha atrás. Lo que gira en ese segundo depende de la aceleración y del lado.

| Parámetro                                                 | Valor  | Qué hace                                                                                              | De dónde sale                                                                 |
|-----------------------------------------------------------|--------|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| V_MIN_MS                                                  | 0,0915 | Velocidad con el comando más chico posible ( $u \rightarrow 0^+$ ), en m/s                            | ■ Medido en banco el 27/08; confirmado en piso                                |
| V_MAX_MS                                                  | 0,3806 | Velocidad con $u = 1$ , en m/s                                                                        | ■ Ídem                                                                        |
| DIAM_RUEDA_M                                              | 0,060  | Diámetro de la rueda                                                                                  | ■ Medido el 24/07                                                             |
| TROCHA_M                                                  | 0,18   | Trocha <b>efectiva</b> : la que reproduce los giros en el lugar. <b>Sólo la usa el simulador</b>      | ■ Con regla son 21,5 cm; la que gira el robot es 18                           |
| PWM_MIN_*, PWM_TOP_*,<br>PWM_BREAKAWAY_*,<br>BREAKAWAY_MS | (piso) | Copia informativa de la calibración del firmware: 300/230, 413/470, 380/280, 150 ms. No los usa nadie | ■ Puestos al día el 01/10 con los de piso. Los que valen están en el firmware |

### Para qué están acá si el Pi no toca el PWM

La ley de control necesita saber **en qué velocidades reales se traduce lo que manda**: de ahí salen V\_LIN\_MAX (el comando que da 0,25 m/s), los registros en m/s y el simulador. Las funciones `velocidad_real()` y `comando_para_velocidad()` hacen esa conversión.

| Parámetro          | Valor   | Qué hace                                                        | De dónde sale                                           |
|--------------------|---------|-----------------------------------------------------------------|---------------------------------------------------------|
| PUERTO_SERIE       | ttyUSB0 | El puerto del ESP32 en el Pi:<br>/dev/ttyUSB0                   | ■ Confirmado el 22/09 (CP2102). Si falla, lo busca solo |
| BAUDIOS            | 115200  | Velocidad del puerto                                            | Igual que en el firmware                                |
| TIMEOUT_SERIE_S    | 0,05    | Cuánto espera el hilo lector por una línea                      | □ Valor inicial                                         |
| PERIODO_COMANDO_S  | 0,10    | Cada cuánto el latido repite el comando (10 Hz)                 | Cinco latidos por ventana del watchdog                  |
| VIGENCIA_COMANDO_S | 0,40    | Edad máxima de un comando para que el latido lo siga repitiendo | ■ Probado: con el lazo colgado, frena en menos de 1 s   |
| WATCHDOG_ESP32_MS  | 500     | Informativo: lo que tarda el ESP32 en frenar solo               | El que vale está en el firmware                         |

### La vigencia tiene que caer entre dos cosas

Tiene que ser **holgada contra el peor frame** ( $\sim 0,1$  s por ciclo; el peor visto, 1,3 s con la imagen rota) y **más corta que el watchdog** (0,5 s). Con 0,40 s, un ciclo normal nunca vence y un cuelgue frena antes de que tenga que actuar el ESP32.

| Parámetro                      | Valor   | Qué hace                                                                         | De dónde sale                                |
|--------------------------------|---------|----------------------------------------------------------------------------------|----------------------------------------------|
| DIST_PARADA_CM                 | 20      | En APROXIMAR: a esta distancia o menos, llegó (si es el oso) o se desvía (si no) | Sin cambios; el robot frena antes por tamaño |
| DIST_OBSTACULO_CM              | 30      | En la fase 3 de HUIR y en el avance de BUSCAR: no avanza, gira                   | ■ Con 20, la lectura bajó a 11 cm            |
| SONAR_RETENCION_S              | 0,3     | Cuánto se sostiene un obstáculo tras la última lectura que lo vio                | ■ Leyó 16, 150, 11 y avanzó con el 150       |
| DIST_INVALIDA                  | −1      | Lo que manda el ESP32 cuando no hay lectura                                      | Parte del protocolo                          |
| SONAR_CENTRADO                 | 0,7     | Con el oso más centrado que esto, lo que ve el sonar “es el oso”                 | ■ Era 0,35; el robot llega descentrado       |
| SONAR_FRACCION_AREA            | 0,6     | Y además el oso tiene que ocupar esta fracción del área objetivo                 | □ Nunca llegó a decidir                      |
| BUZZER_OFF, _HALLAZGO, _ALARMA | 0, 1, 2 | Los patrones del buzzer: suena al huir y al llegar                               | Parte del protocolo                          |
| FPS_OBJETIVO                   | 5,0     | Criterio de aceptación del lazo; main.py avisa si queda debajo                   | Métrica del plan. Medido: ~11                |

## Desde el 2 de octubre, actúan

El ultrasónico está cableado. Los tres valores marcados ■ salieron de **una corrida cada uno**: son ajustes a ojo. Con el sensor desconectado el ESP32 manda −1, que se lee como “no sé”: el robot anda igual, sin ver obstáculos, y main.py lo avisa.

## Si pasa esto, tocar aquello

| Síntoma                                       | Parámetro            | Hacia dónde                      |
|-----------------------------------------------|----------------------|----------------------------------|
| Gira y pasa por delante del oso sin verlo     | BUSCAR_PASO_GIRO_S   | Bajar (pasos más chicos)         |
| Lo ve en la pausa pero no llega a confirmarlo | BUSCAR_PASO_MIRAR_S  | Subir                            |
| La búsqueda es muy lenta                      | BUSCAR_PASO_MIRAR_S  | Bajar, con cuidado               |
| No completa la vuelta antes de avanzar        | BUSCAR_GIRO_S        | Subir                            |
| Al acercarse cruza el oso de un lado al otro  | KP_ANG               | Bajar                            |
| Al acercarse el oso se le va por un costado   | KP_ANG (o V_LIN_MAX) | Subir (o bajar la velocidad)     |
| Frena demasiado lejos / demasiado cerca       | AREA_OBJETIVO        | Subir / bajar                    |
| Llega demasiado rápido y se pasa              | KP_LIN, V_LIN_MAX    | Bajar                            |
| No huye de la mochila                         | UMBRAL_POR_CLASE     | Bajar (mirar antes qué clase ve) |
| Huye de cosas que no son la mochila           | UMBRAL_POR_CLASE     | Subir                            |
| Al huir no queda de espaldas                  | HUIR_GIRO_S          | Subir (gira más)                 |
| Huye dos veces seguidas sin ver nada          | HUIR_REFRACTARIO_S   | Subir                            |

Antes de tocar nada: mirar la línea **Imagen** del análisis. Con la cámara rota, todos los síntomas se parecen.

## Protocolo y seguridad

---

## Del Pi al ESP32

```
M,<v_lin>,<v_ang>    enteros de -100 a 100
B,<patron>            0 apagado, 1 hallazgo,
                     2 alarma
```

## Del ESP32 al Pi (20 veces por segundo)

```
D,<dist_cm>,<enc_izq>,<enc_der>
# ...                diagnostico: se descarta
```

## Una sesión real, resumida

```
<- D,153,0,0        pared a 1,5 m
-> M,0,0             mirando
-> M,0,35            un paso de giro
-> M,0,0             mirando
-> M,55,12           hacia el oso
-> M,0,0             llegado
-> B,1               hallazgo: pitido
```

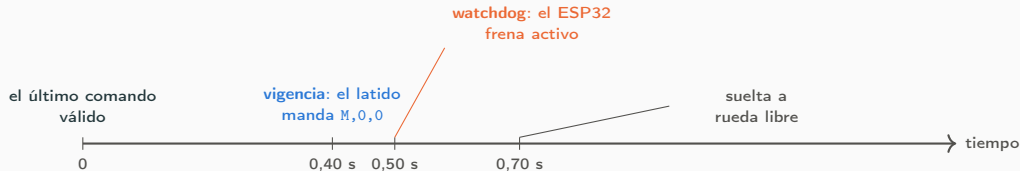
## Por qué en texto

Se lee con cualquier monitor serie y se depura a ojo. A 115200 baudios, una línea de 10 caracteres tarda menos de 1 ms: el protocolo no es el cuello de nada.

## Las reglas chicas que importan

- v\_ang positivo es girar a la **derecha**.
- dist\_cm = -1 es “sin lectura”, **no** “distancia cero”. Tratarlo como cero frenaría el robot de entrada.
- Los encoders se mandan en cero en vez de omitirse: el Pi lee siempre el mismo formato.
- Las líneas que empiezan con # son para humanos; el Pi las cuenta y las tira.
- Para parar, exactamente M,0,0.

# Las dos redes de seguridad, en el tiempo



## Primera red: la vigencia

Actúa si el **programa sigue vivo** pero el lazo de visión dejó de decidir (un frame colgado, la cámara trabada). La pone el Pi.

## Segunda red: el watchdog

Actúa si el Pi **dejó de hablar**: programa muerto, cable desconectado, SSH cortado. La pone el ESP32 y no depende de nadie.

## Tercera: la mano

El conector de 12 V. Es la única que sirve si el robot hace algo *coherente pero equivocado*, como avanzar contra una pared.

Las dos primeras se probaron en hardware el 22 de septiembre, con el robot elevado: dejando el programa vivo pero callado, y matándolo con `kill -9`.

## Dos trampas de instrumentación que ya costaron tiempo

### Abrir el puerto reinicia el ESP32

El puente USB (CP2102) baja la línea DTR al abrir el puerto, y eso reinicia el ESP32, que arranca con los motores frenados.

La primera prueba del watchdog estaba mal por esto: se mataba el programa y se abría el puerto “para ver” si el ESP32 frenaba... y abrirlo **ya lo frenaba**. Un instrumento que altera lo que mide.

### Los 2 segundos del arranque

Por lo anterior, Enlace espera 2 s después de abrir, hasta que el ESP32 termina de reiniciarse.

Si se lanza algo con timeout  $N$ , el movimiento dura  $N - 2$  segundos. Parece una falla del enlace y no lo es.

### Y una mejora que salió del barrido a pasos

`mover()` sólo actualizaba el comando y dejaba que el latido de 10 Hz lo mandara: cada cambio tardaba entre 0 y 0,1 s en salir, al azar. Para avanzar no importa. Para un pulso de giro de 0,25 s es hasta el 40 % en cada punta: un paso giraba la mitad o el doble que el anterior. Ahora **los cambios de comando se mandan en el acto** y el latido sólo sostiene. Como escriben dos hilos, la escritura va con candado.

## Scripts, registros y herramientas

---

| Script                                      | Corre en | Qué hace                                                                                                                                 |
|---------------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------|
| entrar.ps1                                  | PC       | <b>Encuentra el Pi en la red y abre la sesión:</b> prueba el nombre, la última IP conocida, la tabla ARP y, en un hotspot, barre la red. |
| provisionar_pi.ps1                          | PC       | Copia src/ y scripts_pi/ al Pi, les saca los finales de línea de Windows y <b>corre las verificaciones allá</b> . Se puede repetir.      |
| traer_corrida.ps1                           | PC       | Trae a logs/ los registros de las últimas corridas y, con -Video, el video.                                                              |
| verificar_sd.ps1,<br>escribir_cloudinit.ps1 | PC       | Revisan y reparan la tarjeta SD <b>recién grabada, antes del primer arranque:</b> nombre, usuario, SSH, WiFi y país.                     |
| demo.sh                                     | Pi       | <b>Los comandos de todos los días</b> , uno por acción.                                                                                  |
| inventario_pi.sh                            | Pi       | Lista de comprobación del Pi recién instalado.                                                                                           |
| piso/                                       | Pi       | Las pruebas en el piso y los diagnósticos de la cámara y del ultrasónico.                                                                |

### Para operar sin asistencia

Alcanzan tres: entrar, demo.sh y traer\_corrida. Los de la PC se llaman por su .cmd: el .ps1 directo falla desde Google Drive ("no está firmado").

### Escritos con el Pi apagado, estrenados el 2/10

El estreno encontró **tres fallas**, ya corregidas (en correr, firmware y traer\_corrida.ps1). Queda sin probar fondo.

| Comando             | Qué hace                                                                                                                | Probado |
|---------------------|-------------------------------------------------------------------------------------------------------------------------|---------|
| chequeo             | Alimentación, temperatura, cámara, ESP32, las 79 verificaciones y el disco.<br>Termina con <b>LISTO</b> o con qué hacer | ■       |
| imagen              | ¿Los frames llegan enteros? 6 s sin motores; veredicto SANA, DAÑADA A RATOS o ROTA                                      | ■       |
| ver                 | Lo que detecta la cámara, frame a frame, y la foto en logs/vivo.jpg                                                     | ■       |
| fondo               | Sin el oso ni la mochila a la vista: ¿algo de la sala se confunde con ellos?                                            | □       |
| sonar               | ¿Qué distancia mide el ultrasónico? 5 s sin motores; <b>SIN LECTURA</b> si se soltó un cable                            | ■       |
| correr [seg] [nota] | La corrida, con 5 s de espera y límite de tiempo; al terminar, el análisis                                              | ■       |
| grabar [seg] [nota] | Lo mismo, guardando el video de lo que ve el robot, en MP4                                                              | ■       |
| ultima              | Vuelve a mostrar el análisis de la última corrida                                                                       | ■       |
| parar               | Corta una corrida colgada, como un Ctrl+C                                                                               | ■       |
| firmware            | Le pide al ESP32 sus parámetros y los contadores del sensor                                                             | ■       |
| red                 | En qué WiFi está el Pi y qué redes tiene guardadas                                                                      | ■       |
| hotspot "N" "C"     | Guarda el hotspot del celular, con prioridad                                                                            | ■       |
| apagar              | Apaga el Pi ordenadamente                                                                                               | ■       |

Se llaman como `bash ~/TPF/scripts_pi/demo.sh <comando>`, desde una sesión SSH. ■ probado en el robot ■ a medias (parar, sólo sin corrida que parar) □ sin probar allá.

| Script           | Qué hace                                                                                                                                                                                                                                                    | Watchdog  |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| corrida_piso.sh  | Una corrida de main.py con <b>parámetros cambiados sólo para esa corrida</b> (se restauran al terminar, pase lo que pase), más el registro de voltaje y temperatura cada 0,5 s. Con GRABAR=1 guarda el video. Fue el caballito de las dos jornadas de piso. | sí        |
| escalones.py     | Una secuencia fija de comandos M por el protocolo normal. Para ver qué rueda se cala y cronometrar vueltas.                                                                                                                                                 | sí        |
| paso_giro.py     | Da pulsos de giro como los de BUSCAR y <b>mide con la cámara</b> cuánto se corrió la escena entre pausa y pausa.                                                                                                                                            | sí        |
| diag_camara.py   | Compara YUYV quieto, YUYV girando y MJPG girando.                                                                                                                                                                                                           | sí        |
| freno_sonar.py   | Avanza derecho y <b>frena cuando el ultrasónico lee el umbral</b> : informa a cuántos cm mandó frenar y a cuántos quedó. A velocidad 40, 3 cm de inercia.                                                                                                   | sí        |
| umbral_piso.py   | Duty crudo subiendo, por rueda: ¿desde qué duty arranca apoyada?                                                                                                                                                                                            | <b>no</b> |
| cinetico_piso.py | Duty crudo bajando: ¿hasta qué duty sigue girando?                                                                                                                                                                                                          | <b>no</b> |

## Los dos últimos

Usan el modo de calibración del firmware, que **desactiva el watchdog**. Lo devuelven a protocolo al terminar, pero si algo queda girando: 12 V afuera.

| Script            | Qué responde                                                                                                                                                                                                                                       |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| franjas_quieto.py | ¿La imagen llega rota con el robot quieto? Captura unos segundos, cuenta los frames con franjas grises y los que ven el oso, y deja una grilla y tres frames enteros (el peor, el del medio y el mejor). Con SIN_YOLO=1 captura sin correr la red. |
| franjas_crudo.py  | ¿Los frames vienen rotos de la cámara o se rompen en el USB? Captura MJPG crudo con v4l2-ctl con la traza del driver encendida y cruza las franjas de la imagen con los paquetes perdidos que informa el kernel.                                   |
| que_ve.py         | ¿Qué cree YOLO que tiene adelante? Lista <i>todas</i> las clases que propone el modelo, con umbral bajo y a tres tamaños. Así se supo que la mochila es suitcase.                                                                                  |
| sonar.py          | ¿Qué distancia mide el ultrasónico? Lee la telemetría unos segundos y resume. ~370 cm es "libre"; SIN LECTURA es un cable suelto.                                                                                                                  |
| grilla.py         | Una grilla de frames de un video de corrida, uno de cada $N$ , con el motivo del CSV escrito en cada uno.                                                                                                                                          |
| grilla_tiempos.py | Lo mismo, pero con los frames más cercanos a una lista de instantes: <b>qué veía la cámara en cada evento</b> del registro.                                                                                                                        |

### La regla de uso

Antes de una corrida con motores, lo que se pueda contestar sin motores se contesta sin motores: un frame quieto antes que una corrida.

# Cómo se lee el análisis de una corrida

corrida\_20261001\_162944.csv

Parametros: KP\_ANG=0.3 BUSCAR\_GIRO\_S=17.0 AREA\_OBJETIVO=0.35

Lazo: 298 ciclos en 30.1 s -> 9.91 FPS

dt p50 100 ms, p95 107, max 117

Imagen: 0 de 298 frames con franjas, 0 muy danados

Tiempo por estado: ENCONTRADO 15.1 s BUSCAR 12.6 s

APROXIMAR 2.3 s

12.7s BUSCAR -> APROXIMAR objetivo confirmado

15.0s APROXIMAR -> ENCONTRADO llegue (area)

BUSCAR

12.7s: confirmado 0.20 s despues de verlo

ex +0.79 -> +0.73 a los 0,5 s

APROXIMAR (22 frames con el oso)

ex: RMS 0.526, max 0.73

cruces de lado: 1 - objetivo perdido: 0

ENCONTRADO

a los 15.0 s - ar al frenar 0.375 (objetivo 0.35)

ar con el robot quieto: 0.581

## Qué mirar, en orden

1. **Imagen:** si hay franjas, lo demás no mide el comportamiento.
2. **Lazo:** FPS cerca de 10–11 y sin ciclos largos.
3. **Transiciones:** la historia de la corrida.
4. **Buscar:** si lo seguía viendo medio segundo después de confirmarlo.
5. **Aproximar:** cruces de lado (oscila) y pérdidas.
6. **Encontrado:** el área quieto contra el área al frenar dice cuánto se pasó.

Es la segunda corrida completa del 1 de octubre: oso a 1 m, a 90° a la izquierda.

## scripts\_auxiliares/

Cuadernos de análisis: cada uno toma los datos crudos de una medición y reconstruye el análisis completo, con las figuras para el informe. Hay uno:

01\_calibracion\_motores.ipynb (las rectas por rueda de agosto), con cuatro figuras en figuras/. Los datos van embebidos: corre sin hardware.

## presentacion/

La presentación de arquitectura de agosto (58 diapositivas): el diseño, antes de que existiera la mitad del sistema.

## presentacion\_resumen/

Este documento. TPF\_resumen.tex, las secciones en secciones/ y las imágenes en figuras/.

## logs/

Los registros de las corridas: corrida\_\*.csv, .json y .avi; los volt\_\*.txt de tensión y temperatura; y las grillas de frames de cada prueba del 1 de octubre.

## media/

- El video de la demo completa (89 s, con las cajas y el estado dibujados): 2026-10-01\_demo\_completa\_vista\_del\_robot.mp4.
- 2026-10-01\_imagen\_rota/: 22 imágenes de la imagen rota con el robot quieto, con un índice que dice qué es cada una.

## Lo que falta

Fotos del robot terminado y un video filmado desde afuera.

## Historia y mediciones

---

## Quince jornadas (1 de 2): de la idea al lazo completo

| Día | Fecha    | Qué pasó                                                                                                                                                                |
|-----|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0–1 | 22/07    | Alcance y arquitectura. Chasis, motores, L298 y ESP32 montados y cableados.                                                                                             |
| 2   | 24/07    | Primera prueba de motores. Aparecen dos sorpresas: el L298 se queda con <b>2,5 V</b> (no 1,4) y las ruedas no arrancan al mismo PWM.                                    |
| —   | —        | <i>Cinco semanas parado.</i>                                                                                                                                            |
| 3–5 | 27/08    | Firmware definitivo verificado. <b>Calibración de motores</b> : las dos unidades difieren 2,11 veces; una recta por rueda.                                              |
| 6   | 28/08    | Cuaderno de análisis de la calibración, código de visión y presentación de arquitectura.                                                                                |
| 7   | 01–04/09 | Puesta en marcha del Pi 5 sin monitor. El grabador de tarjetas deja la SD sin configurar: nacen los scripts de verificación.                                            |
| 8   | 08/09    | Fuente del Pi resuelta. Se escribe y verifica <b>todo el comportamiento sin hardware</b> (enlace, control, estados, lazo: 40 verificaciones). La cámara CSI no aparece. |
| 9   | 09/09    | Entorno de Python en el Pi. Térmico medido con la carga real. <b>Tamaño de inferencia 320</b> , decidido sin cámara.                                                    |
| 10  | 22/09    | Entra la <b>webcam USB</b> . Lazo completo a 11,2 FPS. Calibración del área. Todo verificado contra el ESP32 real, con el robot elevado.                                |

Los “días” son jornadas de trabajo, no fechas del calendario.

## Quince jornadas (2 de 2): del banco al piso

| Día | Fecha             | Qué pasó                                                                                                                                                                                                                                                                |
|-----|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 11  | 23/09             | <b>Alimentación portátil.</b> La UPS sola no sostiene los 5 V con YOLO corriendo. Solución: dos fuentes separadas.                                                                                                                                                      |
| 12  | 30/09<br>(mañana) | Sin robot (baterías cargando): registro por corrida, analizador, <b>simulador</b> con predicciones, y la guía para el día de la presentación.                                                                                                                           |
| 13  | 30/09 (tarde)     | <b>Primera vez en el piso.</b> Tres problemas, cada uno escondido detrás del anterior: la rueda izquierda se cala, el ruido de los motores rompe la imagen, y girando de corrido la imagen sale movida. Los tres resueltos por separado; ninguna aproximación completa. |
| 14  | 01/10             | <b>Segunda vez en el piso.</b> La imagen se rompe con los motores parados (causa sin aislar). Se mide el paso de giro. Primera aproximación completa, comportamiento completo, huida (cuatro intentos) y dos demos de 90 s. Ultrasónico y buzzer, a la segunda versión. |
| 15  | 02/10             | <b>Entran el ultrasónico y el buzzer.</b> El primer sensor estaba muerto. Tres ajustes en el piso, uno por corrida. Los comandos escritos la noche anterior corren por primera vez.                                                                                     |

### El patrón de las jornadas

Mientras faltó una pieza (la cámara, las baterías), se avanzó con lo que no dependía de ella. La mitad del trabajo de visión se cerró sin cámara, y todo el comportamiento, sin robot.

### La contracara

Cada pieza que llegó destapó algo que las verificaciones sin hardware no podían ver. El piso, más que ninguna: en dos jornadas cambió casi todos los parámetros del comportamiento.

| Tamaño     | FPS         | Veredicto                  |
|------------|-------------|----------------------------|
| 640        | 3,2         | Debajo del objetivo        |
| 480        | 5,2         | Justo en el límite         |
| <b>320</b> | <b>11,3</b> | Más del doble del objetivo |
| 256        | 17,7        | Sobra; resigna alcance     |

FPS de cámara más red, en el Pi 5. El objetivo del plan era 5.

### Lazo completo

|                              |          |
|------------------------------|----------|
| Con estados y enlace real    | 11,2 FPS |
| Midiendo la imagen (franjas) | ~11 FPS  |
| Además grabando video        | ~10 FPS  |
| Costo de estados y enlace    | ~1 %     |

### Latencia

- Edad del frame al leerlo: 36 a 64 ms.
- Inferencia: 88 ms.
- De foto a decisión: ~**150 ms.**

### Alcance con el oso

- Oso impreso de 6,5 cm: ~45 cm.
- Oso de hoja A4 (20 cm): 1,1 m medido el 22/09; en el piso lo confirmó desde ~1,3 m.
- A 2 m no lo ve: lo encuentra cuando la reubicación lo acerca.

### Confianza típica

Oso: 0,85 a 0,95. Mochila (como suitcase): 0,60 a 1 m y 0,32 a 0,41 a 1,2 m.

| Configuración                 | FPS                  | get_throttled                 | Tensión mín. | Veredicto         |
|-------------------------------|----------------------|-------------------------------|--------------|-------------------|
| Fuente de pared               | 11,2                 | 0x50000 (dos eventos pasados) | 4,95 V       | bien, justo       |
| UPS alimentando todo          | 8,83                 | 0x50005 el 43 %               | 4,695 V      | <b>no alcanza</b> |
| Cargador (Pi) + UPS (motores) | 11,08                | 0x0                           | 4,91 V       | bien              |
| Ídem, en el piso, con motores | 9,6 a 10 (con video) | 0x0                           | 4,92 V       | bien              |

### Lo que dijo la UPS

La caída la provoca **la carga de CPU de YOLO**, no los motores: seguía con las ruedas quietas. No es un tema de capacidad (los mAh sobran): es lo que la salida de 5 V sostiene con carga.

### En el piso

En las diez corridas del 1 de octubre, 0x0 en todas las muestras y tensión entre 4,92 y 5,14 V. El Pi no se reinicia al arrancar los motores.

### Temperatura

- YOLO continuo 150 s: meseta en 73–75 °C, sin recortes.
- En el piso, corridas de 30 a 90 s: 55 a 68 °C.
- A 80 °C empieza a recortar.

### Sin medir

La **autonomía** del cargador. La sesión del 1 de octubre duró dos horas y media sin problemas, pero no es una medición.

| Qué                                      | Valor                |
|------------------------------------------|----------------------|
| Velocidad mínima ( $u \rightarrow 0^+$ ) | 0,091 m/s            |
| Velocidad máxima ( $u = 1$ )             | 0,380 m/s            |
| Avance de prueba en el piso              | 57 cm (predicho: 57) |
| Giro con $v_{ang}$ 35, en régimen        | 124°/s               |
| Giro con $v_{ang}$ 60, en régimen        | 165°/s               |
| Trocha con regla                         | 21,5 cm              |
| Trocha efectiva (girando)                | 18 cm                |

| Pulso de giro | $v_{ang}$ | Imagen renovada                          |
|---------------|-----------|------------------------------------------|
| 0,40 s        | 35        | 61 % ( $\sim 32^\circ$ )                 |
| <b>0,25 s</b> | <b>35</b> | <b>35 % (<math>\sim 18^\circ</math>)</b> |
| 0,40 s        | 10        | 39 % ( $\sim 20^\circ$ )                 |
| 0,25 s        | 10        | 22 % ( $\sim 12^\circ$ )                 |

### Dos trochas

La de la regla (entre centros de rueda) es correcta y **no es la que gira el robot**. La efectiva sale de los giros medidos. Y avanzando, el giro realizado es todavía menor: más o menos la mitad.

### Régimen contra arranque

Los 124 y 165°/s se midieron sobre varias vueltas. Un pulso de 0,25 s da  $\sim 18^\circ$ , o sea 72°/s de promedio; y el giro de la huida, 110° en 1,07 s arrancando desde la marcha atrás.

### Cómo se midió el paso

Con la propia cámara: el corrimiento horizontal de la escena entre una pausa y la siguiente, emparejando puntos. Las tandas de 0,40 s tienen pocas mediciones válidas (pared blanca).

## Las métricas del plan: objetivo, medido y pendiente

| Métrica                | Objetivo              | Medido                                                                                                              |   |
|------------------------|-----------------------|---------------------------------------------------------------------------------------------------------------------|---|
| FPS de inferencia      | 5 o más               | 11,3                                                                                                                | ■ |
| FPS del lazo completo  | 5 o más               | 11,2 (~10 grabando)                                                                                                 | ■ |
| Tasa de detección      | 80 % hasta 2 m        | El alcance es ~1,3 m: la métrica, como está escrita, no se puede cumplir                                            | ■ |
| Falsos positivos       | menos de 1 por minuto | Con el umbral de la mochila en 0,20: 0 en 742 frames en un pasillo; 3 frames sueltos en un hall con sillones negros | ■ |
| Éxito de aproximación  | 8 de 10               | Sin serie: llegó en 5 de las 6 corridas con oso (la que falló tenía la imagen rota)                                 | □ |
| Éxito de escape        | 8 de 10               | Sin serie: las dos primeras pruebas no dispararon; después, seis huidas en tres corridas                            | □ |
| Arbitraje de conflicto | 10 de 10              | Una vez: con oso y mochila juntos, huyó                                                                             | □ |
| Tiempo hasta encontrar | registrar             | 3 s de frente; 15 a 17 s a 90° y 1 m                                                                                | ■ |
| Latencia de reacción   | registrar             | 0,2 s de la primera vista a la huida, más ~0,15 s de edad del frame                                                 | ■ |

■ medido   ■ medido a medias o con salvedades   □ faltan las 10 corridas por escenario. **Los porcentajes de éxito no existen todavía:** lo de hoy son pruebas de puesta a punto, no una evaluación.

## Problemas, soluciones y el hilo conductor

---

| Síntoma                                                       | Causa                                                                                      | Qué se hizo                                                                  |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| A los motores les llega menos tensión de la esperada          | El L298 se queda con 2,5 V, no con los 1,4 estimados                                       | Se asumió 9,5 V en bornes y se recalculó todo                                |
| Las ruedas no arrancan al mismo PWM y no giran parejas        | Los dos motores tienen reducciones distintas ( $\sim 2,1:1$ )                              | Una recta de calibración por rueda, con piso y techo propios, en el firmware |
| Con la recta cargada, la derecha se plantaba a comandos bajos | Se usó como piso el corte con cero del ajuste (36), y el duty de calado real era $\sim 98$ | El piso es el duty de la mínima velocidad que la rueda sostiene              |
| En el piso, la rueda izquierda pita y no gira                 | Calibración hecha con las ruedas al aire: la rueda de menos par recibía el menor duty      | Recalibración apoyado: la carga suma un escalón de duty                      |
| Con batería, el FPS cae de 11 a 8,8                           | La salida de 5 V de la UPS se cae con la carga de YOLO                                     | Cargador aparte para el Pi; la UPS sólo para los motores                     |
| El Pi se reinicia en bucle con el cargador                    | Por USB-C falla la negociación de carga rápida                                             | Conectarlo por la salida USB-A                                               |

## Lo que falló (2 de 3): la cámara y la imagen

| Síntoma                                                 | Causa                                                          | Qué se hizo                                                                                         |
|---------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| El Pi no ve la cámara CSI                               | Falla eléctrica (cable adaptador o módulo): nada en el bus I2C | Se descartó por tiempo; entró una webcam USB                                                        |
| El Pi recién grabado no aparece en la red               | El grabador de tarjetas no escribió la configuración           | Scripts que verifican y reparan la SD antes de arrancar                                             |
| Con los motores andando, cero detecciones y 2,3 FPS     | El ruido rompe la transmisión USB; YUYV no tolera pérdidas     | MJPEG, y webcam y ESP32 en controladores distintos                                                  |
| Girando, el oso pasa por delante y no lo ve             | A 124°/s la imagen sale movida                                 | Barrido a pasos: mirar quieto, girar un tramo                                                       |
| Con el robot <i>quieto</i> , 69 de 71 frames en franjas | Paquetes USB perdidos, de a ratos.<br><b>Causa sin aislar</b>  | Cada corrida registra la salud de la imagen; pendiente probar el límite de corriente USB y el cable |
| El entorno de Python pesa 5,6 GB                        | pip trae la versión de torch con CUDA, que el Pi no usa        | Pendiente: reinstalar la versión de CPU                                                             |

| Síntoma                                              | Causa                                                                           | Qué se hizo                                                  |
|------------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------|
| Las huidas se repiten solas (visto sin robot)        | El confirmador sostiene la mochila 5 frames después de dejar de verla           | Período sordo de 2 s tras cada huida                         |
| El robot nunca llegaría a la velocidad máxima (ídem) | La ganancia limitaba antes que el techo                                         | Ganancia y área objetivo calibradas juntas                   |
| main.py revienta al arrancar con la cámara real      | Un formato de texto mal armado, en una línea que sin cámara no se ejecutaba     | Corregido; ninguna verificación podía verlo                  |
| El barrido le pasa de largo al oso                   | Pasos de 32° y pulsos con retardo al azar                                       | Pasos de 18°; los cambios de comando salen en el acto        |
| Llega torcido, o pierde el oso por un costado        | Ganancia de giro mal ajustada; una rueda que se detenía o invertía              | KP_ANG 0,5; giro acotado a 0,7 veces el avance; más despacio |
| No huye de la mochila                                | Es suítcas para el modelo; confianza al borde del umbral; giraba antes de mirar | Alias, umbral propio, frenar al verla, mirar primero         |
| Huye de costado                                      | El giro de media vuelta deshacía el giro del retroceso                          | Las dos fases giran para el mismo lado                       |

Casi todo lo que falló fue **un número correcto usado fuera del contexto donde vale**.

### Qué quiere decir

Ningún número estaba “mal”. La caída del L298 de la hoja de datos es cierta para ese ensayo. La calibración al aire es exacta al aire. La clase backpack es correcta para las fotos con las que se entrenó el modelo. Lo que falla es **llevar el número a otro contexto** sin volver a medir.

Van **veintiún casos** anotados en la bitácora. Las tres diapositivas que siguen los listan. Los primeros son de física; los del medio, de bancos de prueba; los últimos, de software y de diagnóstico.

### Para qué sirve verlo así

Convierte una lista de tropiezos en un método: al lado de cada número, anotar **en qué condiciones se midió**. Es lo que hacen los comentarios de `config.py`, y es la idea que organiza el informe.

|   | El número                  | El contexto donde dejó de valer                                                                                                                                    |
|---|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Caída del L298: 1,4 V      | La de la hoja de datos, en sus condiciones. El módulo real, con estos motores, se queda con 2,5 V.                                                                 |
| 2 | “350 rpm” en la chapa      | Vale para una de las dos unidades. La otra tiene la mitad de reducción.                                                                                            |
| 3 | Piso de la recta: 36       | Es el corte con cero de un ajuste hecho entre duty 211 y 352. Cerca de velocidad cero la fricción no es lineal: el calado real estaba en $\sim 98$ .               |
| 4 | Techo de velocidad: 0,34   | Se dedujo cuando el duty escalaba desde cero. Con el mapa calibrado, el mismo 0,30 m/s es un comando de 72.                                                        |
| 5 | $V\_LIN\_MAX = 72$         | Correcto, pero no actuaba: la ganancia limitaba en 30. Un parámetro que parecía gobernar algo y no llegaba a hacerlo.                                              |
| 6 | $M\_PERDER = 5$            | Existe para no perder el oso por un frame malo. Después de una maniobra que cambia <i>a propósito</i> lo que mira la cámara, sostiene una mochila que ya nadie ve. |
| 7 | “El Pi recorta a los 30 s” | Cierto con cuatro bucles vacíos; falso con YOLO real. No cambió el Pi: cambió la carga con la que se lo interrogó.                                                 |

## Los veintiún casos (8 a 14)

|    | El número                        | El contexto donde dejó de valer                                                                                                                                     |
|----|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8  | Captura: 0,6 ms                  | Es el costo de sacar el frame de la cola, no la <i>edad</i> del frame, que es de 36 a 64 ms.                                                                        |
| 9  | AREA_OBJETIVO = 0,066            | Vale para el oso de 6,5 cm. Con uno de 20 cm sería frenar a 1 m. <i>El primero anticipado en vez de sufrido.</i>                                                    |
| 10 | Área $\propto 1/d^2$             | No se sostiene en todo el rango medido: por eso el área objetivo es un valor medido y no una extrapolación. Y la primera prueba del watchdog alteraba lo que medía. |
| 11 | "5 V / 3 A" y "10 000 mAh"       | La primera es cierta sólo por USB-A. La segunda es capacidad, no lo que la salida sostiene con carga.                                                               |
| 12 | N_CONFIRMAR = 3                  | Supone una cámara quieta. Con el barrido girando no había ni un frame nítido. <i>Anticipado por el simulador, y peor en el piso.</i>                                |
| 13 | Techo de la rueda izquierda: 238 | Iguala velocidades sin carga, y es el equivocado para mover el robot: le da el menor duty a la rueda de menos par.                                                  |
| 14 | Trocha: 21,5 cm                  | Medida con regla, correcta, y no es la que gira el robot: la efectiva es 18 cm.                                                                                     |

|    | El número                                 | El contexto donde dejó de valer                                                                                                                                                      |
|----|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15 | “YUYV funciona”                           | Con el robot elevado y los motores casi sin carga. En cuanto empujan, deja de funcionar.                                                                                             |
| 16 | “El ruido de los motores rompe la imagen” | Un diagnóstico cierto para lo que se midió ese día, escrito como <i>la</i> causa. Al día siguiente la imagen se rompía con los motores parados.                                      |
| 17 | La clase backpack                         | Correcta para mochilas en una espalda, vistas a la altura de una persona. Apoyada en el piso y vista desde abajo, es una valija.                                                     |
| 18 | Giro: $165^{\circ}/s$                     | Es la velocidad de régimen, medida sobre cuatro vueltas. Una maniobra de un segundo que arranca desde la marcha atrás dio $110^{\circ}$ .                                            |
| 19 | Trocha efectiva: 18 cm                    | Medida girando en el lugar. Avanzando, el giro realizado es más o menos la mitad.                                                                                                    |
| 20 | “60 verificaciones pasan”                 | Una de ellas exigía que la mochila no frenara el barrido, y era la razón por la que el robot no huía. Verificar dice que el código hace lo decidido, no que la decisión fuera buena. |
| 21 | “0 falsas alarmas en 742 frames”          | En un pasillo. Una hora después, en un hall con sillones negros, aparecieron tres. <i>Anticipado y confirmado el mismo día.</i>                                                      |

### Una capa por vez

Escalones de comando antes que el comportamiento; duty crudo antes que el protocolo; un frame quieto antes que una corrida. En el piso, cada arreglo destapó el siguiente: mientras la rueda no giraba era imposible ver que la cámara se rompía.

### Lo que no depende de la pieza que falta

Ante cada bloqueo, preguntar qué parte del problema se puede resolver igual. El comportamiento se verificó sin robot, y el tamaño de inferencia se decidió sin cámara.

### ... y anotar la deuda

Trabajar sin la pieza adelante, pero deja algo sin probar. Conviene escribir *qué* mientras se la contrae.

### Medir la entrada barata antes de creerle a la salida cara

La predicción más alarmante del simulador (27 % de éxito) era la más sensible a un número estimado de una foto. Con la trocha medida daba 78 %.

### Un instrumento puede alterar lo que mide

Abrir el puerto para “mirar” reinicia el ESP32. La carga sintética calienta más que la real. Cada herramienta de medición tiene su propio contexto.

### Pruebas con motores: anunciadas

Cada corrida se describe antes (qué va a hacer el robot, cuánto espacio necesita) y se lanza sólo con el “listo” de quien está al lado, con la mano en los 12 V.

La jornada final: 1 de octubre, en el piso

---

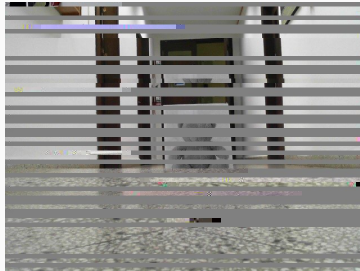
| Hora  | Prueba                     | Qué pasó                                                                                                          |
|-------|----------------------------|-------------------------------------------------------------------------------------------------------------------|
| 15:21 | Aproximación 1 (oso a 90°) | 25 s buscando, <b>cero detecciones</b> . Le pasó de largo al oso.                                                 |
| 15:28 | (sin motores)              | Con el robot quieto, 69 de 71 frames llegan en franjas.                                                           |
| 16:02 | Aproximación de frente     | <b>Primera aproximación completa:</b> 3,1 s. Queda a 37 cm (cinta).                                               |
| 16:16 | Paso de giro (20 pulsos)   | El paso de 0,40 s renueva el 61 % de la imagen; el de 0,25 s, el 35 %.                                            |
| 16:23 | Comportamiento completo    | Busca, encuentra, se acerca, frena. Termina torcido.                                                              |
| 16:29 | Comportamiento completo    | Ídem, bien centrado. Queda a ~28 cm.                                                                              |
| 16:40 | Huida 1                    | No huye: vio la mochila dos frames y el barrido giró igual.                                                       |
| 16:43 | Huida 2                    | No huye: la confianza de la mochila estaba al borde del umbral.                                                   |
| 16:50 | Huida 3                    | <b>Huye</b> , pero gira ~110° y se va de costado.                                                                 |
| 16:54 | Huida 4                    | Huida completa. Gira algo más de media vuelta.                                                                    |
| 17:07 | Demo completa 1 (90 s)     | Oso a 2 m: tarda 36 s en verlo, lo pierde una vez, llega a los 65 s.                                              |
| 17:17 | Demo completa 2 (90 s)     | Cuatro huidas ante la mochila real, tres pérdidas por mover el oso a mano, llegada final. <b>Es la del video.</b> |

Entre corrida y corrida se cambió código: al final del día había 60 verificaciones (eran 54), ocho parámetros con otro valor y dos nuevos.

## La imagen rota (1 de 2): qué se veía



Sobre el escritorio: sana. El oso con confianza  
0,91 en todos los frames.



En el piso, robot quieto y 12 V desenchufados:  
37 % de las filas en franja. El oso está detrás.



El peor frame de esa medición: 63 % de las  
filas.

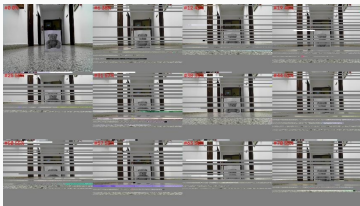
### Qué son las franjas

Con MJPG, cuando se pierde un paquete USB falta un tramo del JPEG, y el decodificador rellena con gris hasta el próximo marcador de reinicio. El frame llega “entero” pero YOLO no reconoce nada detrás.

### Por qué importó tanto

La primera corrida no vio el oso, y la explicación a la vista era el paso de giro. Mirar un frame quieto antes de tocar el paso evitó “arreglar” el barrido con la cámara ciega.

## La imagen rota (2 de 2): qué se descartó y qué no



15:28 — 69 de 71 frames rotos.



15:52 — 0 de 606. Nadie tocó los cables.

### Descartado con mediciones

- El PWM de los motores: pasa con los motores parados.
- La fuente de 12 V: igual con la UPS desenchufada.
- La carga del procesador: se rompe capturando sin YOLO.
- La alimentación del Pi: 0x0, 4,97 a 5,14 V.
- El ancho de banda USB: negocia el máximo y usa la octava parte.
- La cámara en sí: en los ratos buenos, 150 de 150 frames sanos.

### Lo que sí se sabe

Son **paquetes USB perdidos**: con la traza del driver, el kernel registra cientos en los ratos malos y ninguno en los buenos.

### Lo que no se sabe: la causa

Quedan el cable o conector de la webcam y el límite de 600 mA de los USB. El problema **desapareció solo** antes de poder aislarlo. Puede volver.

# Medir el paso de giro con la propia cámara



## Qué muestra

Un frame por pausa, con el robot quieto, a lo largo de 20 pulsos de giro en cuatro tandas. Comparando cada frame con el anterior (emparejando puntos de la mitad de arriba, que es lo lejano) sale cuántos píxeles se corrió la escena.

## Qué se sacó

- El paso vigente renovaba el 61 % de la imagen: demasiado.
- El oso reaparece en el mismo lugar tras 15 pasos y pico: una vuelta son  $\sim 4400$  px, o sea un campo visual de  $\sim 52^\circ$ .

# El comportamiento completo, visto desde el robot



## Qué se ve (corrida de las 16:29)

Arriba a la izquierda, el último paso del barrido. El oso entra por el borde derecho, el robot se queda quieto ("posible objetivo"), lo confirma y avanza. En las filas siguientes el oso va hacia el centro mientras crece, hasta que el robot frena y queda detenido.

## Los números de esa corrida

15 pasos de barrido; confirmado 0,2 s después de verlo; 2,3 s de aproximación; el error lateral bajó de +0,73 a cero en 2 s sin cruzarse de lado. Cada cuadro lleva el frame, el tiempo y el motivo del comando.

# La mochila que resultó ser una valija



Lo que ve el robot: la mochila a 1 m, desde el piso.

Para YOLOv8n, suitcase con confianza 0,60.

## Cómo se supo

La cámara la veía perfecta y `demo.sh ver` decía “nada”. `que_ve.py` le preguntó al modelo por *todas* las clases: suitcase en 15 de 15 frames, backpack en ninguno.

## Por qué

El modelo aprendió mochilas colgadas de una espalda, vistas a la altura de una persona. Apoyada en el piso y mirada desde abajo, con la tapa enrollada, se parece más a una valija.

## Qué se hizo

- **Alias:** suitcase cuenta como la amenaza.
- **Umbral propio de 0,20:** a 1,2 m la confianza baja a 0,32–0,41, justo en el umbral general de 0,35.

## El costo

Con 0,20, objetos oscuros grandes dan frames sueltos de falsa alarma: un marco de puerta, unos sillones negros. Uno solo no dispara nada (hacen falta 3 seguidos), pero hay que mirar el fondo de la sala.

# La huida, vista desde el robot



## Qué se ve (corrida de las 16:54)

Primera fila: retrocediendo, con la mochila alejándose y corriéndose hacia un borde (el robot gira mientras retrocede). Segunda fila: el giro en el lugar, con la imagen movida. De la tercera en adelante: avanzando, ya de espaldas, hasta quedar cerca de una pared.

## Los cuatro intentos

1. No frenaba al ver la mochila sin confirmar.
2. Confianza al borde del umbral; giraba antes de mirar.
3. La fase 2 giraba al revés que la 1:  $\sim 110^\circ$ .
4. Bien. Con 1,3 s se pasó de media vuelta; quedó en 1,2 s.

# La demo completa (la del video)



## Qué se ve

Momentos de la corrida de 90 s, en un espacio abierto. Arriba: la mochila enfrente al arrancar (huye), y alguien moviendo el oso. Abajo: la mochila puesta junto al oso (caja naranja, cortada por el borde) dispara la huida aunque el robot vaya hacia el oso; después la mochila sola, y la llegada final.

## Cómo leer esa corrida

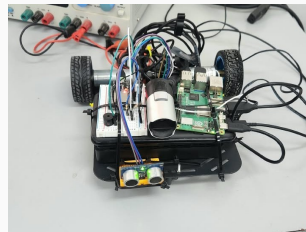
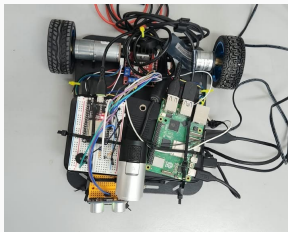
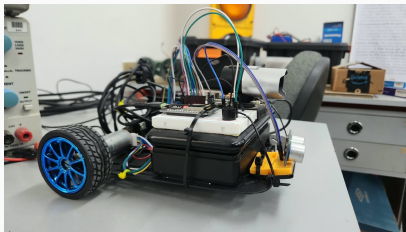
Las cuatro huidas fueron ante la mochila real. Las tres veces que “perdió” el oso, alguien lo estaba moviendo con la mano. Llegó a ENCONTRADO a los 85 s, a unos 26 cm. El video está en media/.

| Qué                         | Antes         | Después                  | Por qué                                      |
|-----------------------------|---------------|--------------------------|----------------------------------------------|
| BUSCAR_PASO_GIRO_S          | 0,40          | 0,25                     | El paso renovaba el 61 % de la imagen        |
| BUSCAR_GIRO_S               | 9             | 17                       | Una vuelta entera con pasos de $18^\circ$    |
| Orden del barrido           | gira, mira    | mira, gira               | No apartar la vista antes de mirar           |
| Frenar al ver               | sólo el oso   | oso y mochila            | La mochila no llegaba a confirmarse          |
| Envío de comandos           | con el latido | en el acto               | Los pulsos cortos salían con retardo al azar |
| KP_ANG                      | 0,8           | 0,5                      | 0,8 se pasaba; 0,3 perdía el oso             |
| Giro al avanzar             | sin límite    | $\leq 0,7 \cdot v_{lin}$ | Ninguna rueda se detiene ni se invierte      |
| V_LIN_MAX                   | 72            | 55                       | Más tiempo para centrar                      |
| AREA_OBJETIVO, KP_LIN       | 0,222 y 4,5   | 0,35 y 2,4               | Frenar a 30 cm en vez de a 50                |
| Clase de la mochila         | backpack      | más suitcase             | Así la ve el modelo desde el piso            |
| Umbral de la mochila        | 0,35          | 0,20                     | A 1,2 m da 0,32–0,41                         |
| Fase 2 de HUIR              | a la derecha  | como la fase 1           | Deshacía el giro del retroceso               |
| HUIR_GIRO_S                 | 1,1           | 1,2                      | 1,1 daba $110^\circ$ ; 1,3 se pasaba         |
| Búsqueda tras perder el oso | a la derecha  | hacia donde lo vio       | Tardó 23 s en reencontrarlo                  |
| Registro                    | —             | columna franjas          | Saber si la cámara estaba ciega              |

**El 2 de octubre: ultrasónico y buzzer**

---

# El robot terminado



## Qué se ve

- Al frente, sobre una protoboard amarilla, el **ultrasónico** (los dos “ojos”), y arriba la **webcam**.
- En la protoboard blanca, el **ESP32**, el divisor del sensor y el **buzzer** (el cilindro negro).
- Al lado, la **Raspberry Pi 5**; atrás, el **L298N** (rojo) y los dos motores.
- Debajo, la caja negra con las baterías.

## Qué cambió respecto del 1 de octubre

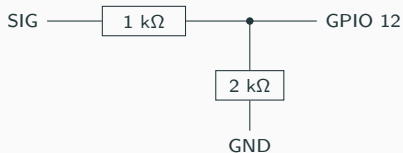
El día anterior se había decidido dejar el ultrasónico y el buzzer para una segunda versión, por tiempo. El 2 de octubre se **revirtió**: la idea era agregar el ultrasónico “en media hora”, para que el robot no chocara contra las paredes al huir.

## Lo que llevó

Algo más de **dos horas**, y entró también el buzzer. Las fotos están en Fotos del bot/.

# Cómo quedó cableado

| Señal       | GPIO | Nota                                        |
|-------------|------|---------------------------------------------|
| PING))) SIG | 12   | Con divisor. Disparo y eco por el mismo pin |
| PING))) 5V  | VIN  | 5 V del USB                                 |
| PING))) GND | GND  | Común con el ESP32                          |
| Buzzer +    | 14   | Activo, de dos patas, directo               |
| Buzzer -    | GND  |                                             |



## Por qué el divisor

El sensor contesta a **5 V** y el ESP32 no los tolera. Hacia el ESP32 baja a 3,3 V; hacia el sensor, el disparo llega a 3,3 V por la de 1 kΩ, que alcanza (la entrada es TTL). **Cruzadas**, el eco llega a 1,7 V y el sensor parece mudo.

## Por qué esos pines

El plan era el 19 para el sensor y el 4 para el buzzer. En la protoboard sólo queda accesible **una fila** del ESP32, y de esa fila los únicos libres que pueden ser salida son el 12 y el 14.

## Dos trampas de pines

- El **35** es de sólo entrada: el buzzer se conectó ahí primero, y ahí no suena.
- El **12** es pin de arranque: en alto al salir del reset, el ESP32 no arranca. La de 2 kΩ lo tiene en bajo.

## Qué había y qué hizo falta

El firmware estaba escrito para un HC-SR04, con un pin para disparar y otro para el eco. El **Parallax PING)))** usa uno solo: el ESP32 manda el pulso de disparo y enseguida escucha por el mismo cable.

## Cómo se hace

1. El pin pasa a **salida** y da un pulso de  $5\ \mu\text{s}$ .
2. Pasa a **entrada**. El sensor espera  $750\ \mu\text{s}$  antes de contestar: sobra tiempo.
3. Una interrupción mide el ancho del pulso de eco:  $58\ \mu\text{s}$  por centímetro.

La interrupción sólo mira dentro de la **ventana de escucha**: así el propio disparo nunca se toma por un eco.

## Lo que hay que saber de las lecturas

- $\sim 370\ \text{cm}$  es “**libre**”: sin nada adelante el sensor devuelve su pulso más largo. No es una distancia.
- $-1$  es “**no contestó**”: sensor desconectado. El Pi lo lee como “no sé”, no como distancia cero.
- Mide de 2 cm a 3 m, cada 60 ms.

## Lo que quedó en el firmware

- PIN\_SONAR 12, PIN\_BUZZER 14, BUZZER\_PASSIVE 0.
- Tecla e: contadores del sensor (disparos, flancos, ecos).
- Tecla u: un disparo y la línea mirada con el ADC.
- Tecla U: línea en alto fijo, para medirla con el tester.

## El primer sensor estaba muerto: cómo se descartó todo lo demás

| Prueba                                                              | Qué descartó                                                                |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Contadores de disparos, entradas a la interrupción y ecos           | La interrupción: ve exactamente 2 flancos por disparo, los del propio pulso |
| Leer el pin con el pull-up interno puesto                           | Que el divisor no estuviera en ese pin: lee 0, hay camino a GND             |
| La línea mirada con el ADC durante 25 ms después del disparo        | Un eco a media altura por las resistencias cruzadas: 0,00 V                 |
| Disparos de 5, 20, 100 y 1000 $\mu$ s                               | El ancho del disparo                                                        |
| Línea en alto fijo, medida con el tester sobre los pines del sensor | El cableado: 3,3 V en SIG y 5 V en la alimentación                          |

### El resultado

Con todo eso bien, el sensor seguía mudo. Cambiado por otro igual: **30 cm clavados, 120 de 120 lecturas**, con una caja a 30 cm. Costó unos 20 minutos.

### Por qué se hizo así

Con todas las lecturas en  $-1$  no se sabe si falla el firmware, el cableado o el sensor. Cada prueba se agregó al firmware para descartar una causa **sin pedirle al usuario que revisara a ciegas**. El procedimiento quedó en el `RUNBOOK.md`, Paso 25.

## Dos umbrales

- **DIST\_PARADA\_CM = 20**: el de *llegada*. En APROXIMAR, algo a menos de 20 cm cuenta como llegada si es el oso; si no, se desvía.
- **DIST\_OBSTACULO\_CM = 30**: el de *esquivar*. En la fase 3 de HUIR y en el pulso de avance de BUSCAR: no avanza, **gira en el lugar**.

Al oso hay que poder acercarse; a una pared, no.

## Retención: 0,3 s

Un obstáculo se da por cierto hasta 0,3 s después de la última lectura que lo vio (SONAR\_RETENCION\_S). Girando frente a una pared el eco se pierde de a ratos y aparece un “libre” falso.

## ¿El eco es el oso?

Si el oso está a la vista con  $|error_x| < 0,7$  y ya ocupa el 60 % del área objetivo, lo que hay adelante es él. El 0,7 sale del cono del sensor ( $\pm 20^\circ$ ) contra el semicampo de la cámara ( $26^\circ$ ).

## Lo que el sensor no arregla

- **Sólo mira adelante**. El retroceso y el giro de la huida siguen a ciegas: las lecturas más bajas del día (7 y 13 cm) fueron al terminar el giro, no avanzando.
- **Una pared muy de costado no la ve** hasta estar encima: el eco rebota para otro lado.
- **Sin lectura, el robot anda igual pero a ciegas**. `main.py` lo avisa al arrancar.

| Dónde | Prueba                    | Qué pasó                                                                                  |
|-------|---------------------------|-------------------------------------------------------------------------------------------|
| banco | Primer sensor             | Todas las lecturas en $-1$ . Cinco pruebas después: el sensor está muerto.                |
| banco | Segundo sensor            | 30 cm con una caja a 30 cm, 120 de 120.                                                   |
| piso  | Quieto, pared a 1,5 m     | 153 cm, estable. Sin ecos del piso.                                                       |
| piso  | Avance hasta leer 20 cm   | A velocidad 40: manda frenar leyendo 19 y queda a <b>16 cm</b> .                          |
| 10:34 | Huida, pared a 1 m        | El avance terminó solo a 27 cm: el sensor no llegó a actuar.                              |
| 10:36 | Huida, pared a 80 cm      | <b>Esquiva</b> : corta a 18 cm y gira. Después lee 16, <b>150</b> , 11 y avanza un ciclo. |
| 10:41 | Demo del oso              | Se acerca sin desvíos falsos (119 a 28 cm) y frena por tamaño.                            |
| 10:54 | (cable del sensor suelto) | <b>553 filas sin lectura</b> : 52 s a ciegas, cortada a mano.                             |
| 10:59 | Con dos umbrales y buzzer | Corta el avance a <b>29 cm</b> ; no baja de 27. Suena en los tres momentos.               |
| 11:07 | Grabada (la del video)    | Huye a los 2 s, esquiva a 29 cm, encuentra el oso a los 8 s.                              |

Seis corridas con el programa completo, **3675 frames y ninguno con franjas**. Al mediodía: 69 verificaciones (eran 60); a la tarde, 79.

# Tres ajustes, cada uno por una corrida

## 1. El oso descentrado

Al llegar, oso grande pero corrido a la derecha (+0,53) y el sensor en 22 cm. Con 20 lo habría tomado por un obstáculo ajeno y habría girado a tope para el otro lado.

SONAR\_CENTRADO: **0,35** → **0,7**. La regla pedía un oso más centrado de lo que el sensor distingue.

## 2. La lectura suelta

Girando frente a la pared: 16 cm, **150**, 11. Con el 150 avanzó un ciclo contra la pared.

SONAR\_RETENCION\_S = **0,3**. En la corrida siguiente apareció un 172 entre lecturas de 12–16 y el robot no avanzó.

## 3. Poco margen

Con un solo umbral de 20 cm, en las huidas la lectura bajó a 16, 11 y 12 cm: el robot no frena, pivotea, y el frente barre un arco.

DIST\_OBSTACULO\_CM = **30**, aparte del de llegada. Corta a 29 y no baja de 27.

## Los tres valen por una sola corrida

Son ajustes a ojo, hechos sobre lo que mostró el registro. Ninguno tiene una serie de mediciones detrás.

## El estreno de los comandos de la noche anterior

- `demo.sh` correr, con la webcam desenchufada, mostraba el análisis de la corrida **del día anterior** como si fuera el de ésta. Ahora dice que no se hizo.
- `traer_corrida.ps1` no copiaba nada: la ruta tiene espacios y terminaba en barra invertida. Y elegía el `.avi` en vez del `.mp4`.
- `demo.sh` firmware, en su primer uso tras encender: “comando desconocido”.

Los tres corregidos. El hotspot se cargó y se probó al cierre: el Pi arranca en él. Sin estrenar: fondo.

## Una corrida a ciegas

Se soltó un cable del sensor y nadie lo supo hasta ver el robot. Y al cancelar desde la PC, **el programa siguió andando en el Pi**. Quedaron el aviso de `main.py` al arrancar y `demo.sh` sonar.

## Bajo voltaje con las baterías cargándose

Con las baterías enchufadas a sus cargadores, el Pi marcó bajo voltaje en reposo: 4,72–4,80 V. Desconectadas: 5,10 V y 0x0. Visto dos veces el mismo día.

## La webcam se había soltado

Al bajar el robot al piso, el Pi dejó de verla. Reinsertada: cero franjas en 3675 frames.

## Qué dice eso de la imagen rota del 1 de octubre

Que la causa puede haber sido el enchufe o la alimentación. Pero ese día cambiaron **las dos cosas a la vez**, así que no se sabe cuál. Sigue sin aislar.

| Qué                             | Antes                | Después                         | Por qué                                    |
|---------------------------------|----------------------|---------------------------------|--------------------------------------------|
| Ultrasónico                     | sin cablear          | PING))) en el GPIO 12           | Que no avance contra una pared             |
| Buzzer                          | sin cablear          | activo, en el GPIO 14           | Avisar la huida y la llegada               |
| Firmware, sensor BUZZER_PASSIVE | HC-SR04, 2 pines 1   | un solo pin 0                   | El sensor que había<br>El buzzer es activo |
| Pulso de avance de BUSCAR       | avanza siempre       | gira si hay algo                | Era el único avance sin mirar              |
| DIST_OBSTACULO_CM               | —                    | 30                              | Con 20, la lectura bajaba a 11 cm          |
| SONAR_RETENCION_S               | —                    | 0,3                             | Una lectura libre suelta hacía avanzar     |
| SONAR_CENTRADO                  | 0,35                 | 0,7                             | El robot llega descentrado                 |
| main.py                         | —                    | avisa si no hay lectura         | Una corrida entera salió a ciegas          |
| demo.sh                         | 12 comandos          | 13: sonar                       | Ver el sensor sin mover el robot           |
| Scripts de piso/                | —                    | sonar.py,<br>freno_sonar.py     | Medir quieto, y dónde queda al frenar      |
| Barrido después de huir         | siempre a la derecha | sorteado, o lejos de la mochila | Huía en ciclo: seis veces en un minuto     |
| Verificaciones                  | 60                   | 79                              | Los casos nuevos del sensor y del barrido  |

El detalle está en `PLAN.md` §13, Día 15, y en `HARDWARE.md` §9.

## Operación, estado y pendientes

---

## Cómo se opera, en siete pasos

1. **Armar.** Cargador → Pi por **USB-A**, sin enchufar a la pared. Webcam **firme** en el USB azul de abajo, ESP32 en el negro de abajo. UPS de 12 V **desenchufada**. Notebook y Pi en el mismo hotspot.
2. **Entrar.** Desde PowerShell, en la carpeta del proyecto: `.\scripts_pi\entrar.cmd` (busca el Pi y abre la sesión).
3. **Chequear.** `bash ~/TPF/scripts_pi/demo.sh` chequeo: tiene que terminar en **LISTO**.
4. **Medir la imagen y el sensor.** `demo.sh imagen: imagen SANA`. `demo.sh sonar`: una distancia creíble, no **SIN LECTURA**.
5. **Mirar.** `demo.sh ver con el oso a 1 m: teddy bear`. Con la mochila: `backpack`. Y `demo.sh fondo` sin ninguno.
6. **Correr.** Enchufar los 12 V, robot en el piso, y `demo.sh correr 60 "nota"` (o grabar, con video). Tarda 10 a 15 s en arrancar; durante la cuenta, sacar las manos.
7. **Parar.** Ctrl+C. Emergencia: sacar los 12 V, no el Pi.

### Al terminar

Sacar los 12 V, traer el video con `traer_corrida.cmd -Video`, `demo.sh apagar`, y esperar a que el LED verde deje de parpadear.

### El manual para hacerlo sin ayuda

GUIA\_PRESENTACION.md y su PDF (12 páginas): cada paso con lo que tiene que salir, cómo leer la pantalla, problemas, ajustes con vuelta atrás y plan B.

# Conectarse al Pi y trabajar con él

## Entrar

```
.\scripts_pi\entrar.cmd          # lo busca y entra
.\scripts_pi\entrar.cmd -Ip 192.168.43.57    # con la IP
ssh -i ~/.ssh/id_ed25519_pi admin@IP        # a mano
```

## Copiar cambios de la PC al Pi, y traer resultados

```
.\scripts_pi\provisionar_pi.cmd      # copia y verifica alla
.\scripts_pi\traer_corrida.cmd -Video  # registros y video
```

## En el Pi, a mano

```
cd ~/TPF/src && source ../.venv/bin/activate
python prueba_estados.py          # las 60, todas "ok"
python main.py --simular --espera 0 --duracion 10
python analizar_corrida.py ../logs/corrida_*.csv
```

## Una corrida de prueba con video y voltaje

```
cd ~/TPF/scripts_pi/piso
GRABAR=1 bash corrida_piso.sh etiqueta 30 "nota" KP_ANG=0.4
```

## El ciclo de trabajo

1. Editar en la PC.
2. Correr prueba\_estados.py en la PC.
3. Copiar al Pi y correrla allá.
4. Comparar que el código sea el mismo en los dos lados.
5. Probar sin motores (--simular, demo.sh ver).
6. Recién entonces, la corrida.

## La red

En la sala no va a estar la WiFi de siempre. Hay que **cargar el hotspot del celular en el Pi** antes, con demo.sh hotspot: sin eso no hay forma de entrar. Pendiente.

| Bloque                                                           | Estado al 2 de octubre                                                                                                               |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <input checked="" type="checkbox"/> Mecánica, cableado, driver   | Hecho y verificado                                                                                                                   |
| <input checked="" type="checkbox"/> Firmware del ESP32           | Verificado; calibración de piso, ultrasónico de un pin y buzzer                                                                      |
| <input checked="" type="checkbox"/> Tracción                     | Avanza derecho y gira; rango 0,091 a 0,380 m/s                                                                                       |
| <input checked="" type="checkbox"/> Alimentación portátil        | Dos fuentes; 0x0 en todas las corridas                                                                                               |
| <input checked="" type="checkbox"/> Raspberry Pi y entorno       | Funcionando; 12 GB libres                                                                                                            |
| <input checked="" type="checkbox"/> Visión                       | 10,6 FPS y detecta. La imagen rota no volvió el 2/10; <b>causa sin aislar</b>                                                        |
| <input checked="" type="checkbox"/> Máquina de estados y control | Funcionando en el piso; 79 verificaciones                                                                                            |
| <input checked="" type="checkbox"/> Pruebas integradas           | Demo hecha, con video. Faltan las 10 corridas por escenario                                                                          |
| <input checked="" type="checkbox"/> Operación                    | Guía para operar sin asistencia; hotspot cargado y probado; casi todos los comandos ya corrieron en el robot. <b>Falta el ensayo</b> |
| <input checked="" type="checkbox"/> Ultrasónico y buzzer         | Cableados y probados en el piso el 2 de octubre                                                                                      |
| <input type="checkbox"/> Encoders                                | Fuera del alcance: segunda versión                                                                                                   |
| <input type="checkbox"/> Informe                                 | Sin empezar. Material: la bitácora, el cuaderno, este documento                                                                      |

☒ hecho    ☒ funciona, con algo pendiente    ☐ sin hacer

## Antes de la presentación

1. **Ensayar con la guía**, sin ayuda, por el hotspot (ya cargado y probado: el Pi arranca en él). Ahí se estrena fondo.
2. **Antes de cada corrida**: baterías fuera del cargador, webcam firme, y `demo.sh` sonar.
3. **La imagen rota**: si vuelve, Paso 24 del RUNBOOK.
4. En la sala: `demo.sh` fondo, por las falsas alarmas, y decidir dónde va la mochila.

## Para el informe

7. **Las 10 corridas por escenario** (Paso 23 del RUNBOOK): aproximación, huida, conflicto y sala vacía.
8. **Autonomía** del cargador.
9. Un **video desde afuera**, con sonido. Las fotos ya están.
10. Escribirlo. El hilo conductor ya tiene veintiún casos.

## Ajustes finos que quedaron a ojo

- Los tres del ultrasónico: una corrida cada uno.
- El giro de la huida (1,2 s; depende del lado).
- Al frenar se tuerce a la derecha: queda con el oso  $\sim 9^\circ$  a la izquierda.
- La pausa del barrido (0,6 s): la vuelta tarda 17 s.

Cómo tocarlos, con vuelta atrás: guía, sección 11.

### Lo que ya está previsto en el código

- **Encoders:** pines reservados; el protocolo ya lleva los dos campos.

### Lo que dejó de ser segunda versión

El **ultrasónico** y el **buzzer** entraron el 2 de octubre. Lo que el código tenía previsto desde septiembre anduvo casi sin cambios; lo que hubo que agregar salió del piso: dos umbrales, la retención, y el avance de BUSCAR.

### Lo que el ultrasónico mostró que falta

**Sensores atrás y a los costados.** Hoy el robot retrocede y gira a ciegas, y una pared muy de costado no la ve hasta estar encima.

### Lo que el piso mostró que haría falta

- **Frenar más cerca:** con el ultrasónico ya puesto se podría llegar a 10 cm con un avance final guiado por la distancia; con la cámara sola el oso se sale del cuadro.
- **Lazo de velocidad con encoders:** corregiría la torcedura al acelerar y al frenar, que hoy carga la ley de giro.
- **Más alcance:** inferir a mayor tamaño sólo mientras busca (está quieto, el FPS importa menos).
- **Mitigar el ruido:** capacitores en los motores, ferrita en el cable de la webcam, cables trenzados.
- **Volver de ENCONTRADO** a buscar después de unos segundos.

| Término                      | Qué quiere decir acá                                            |
|------------------------------|-----------------------------------------------------------------|
| Frame                        | Una imagen de la cámara. El lazo procesa $\sim 11$ por segundo. |
| Caja ( <i>bounding box</i> ) | El rectángulo con el que la red marca un objeto.                |
| Confianza                    | Qué tan segura está la red de una caja, de 0 a 1.               |
| COCO                         | El conjunto de 80 clases con el que viene entrenado el modelo.  |
| YOLOv8n                      | La red de detección; "n" de <i>nano</i> , la más chica.         |
| Inferencia                   | Pasar un frame por la red. Cuesta 88 ms.                        |
| Tamaño de inferencia         | El lado de la imagen que ve la red (320 px).                    |
| error_x                      | Posición lateral del objeto en la imagen, de $-1$ a $+1$ .      |
| area_ratio                   | Fracción de la imagen que ocupa la caja: tamaño aparente.       |
| Confirmación temporal        | Pedir 3 frames seguidos antes de crearle a una detección.       |
| Período sordo                | Los 2 s posteriores a una huida, en que se ignora la mochila.   |
| Arbitraje                    | Decidir qué conducta gana cuando compiten dos.                  |
| MJPEG / YUYV                 | Formatos de video de la webcam: comprimido y sin comprimir.     |
| Franjas                      | Tramos grises de un frame MJPEG al que le faltan datos.         |

| Término               | Qué quiere decir acá                                                                |
|-----------------------|-------------------------------------------------------------------------------------|
| PWM y duty            | Encender y apagar el motor muy rápido; el duty es la fracción encendida (0 a 1023). |
| Puente H              | El circuito del L298 que permite invertir el sentido del motor.                     |
| Tracción diferencial  | Dos ruedas motrices; se gira dándoles distinta velocidad.                           |
| Trocha                | Distancia entre las dos ruedas.                                                     |
| Zona muerta (motor)   | El rango de duty bajo en que la rueda no se mueve.                                  |
| Zona muerta (control) | El rango de error chico en que la ley no gira.                                      |
| Pulso de arranque     | Duty extra durante 150 ms para vencer la fricción estática.                         |
| Calarse               | Que la rueda reciba duty y no gire (el motor pita).                                 |
| Watchdog              | El temporizador del ESP32 que frena si no llegan órdenes.                           |
| Latido                | El hilo que repite el comando al ESP32 diez veces por segundo.                      |
| Vigencia              | La edad máxima de un comando para que el latido lo repita.                          |
| get_throttled         | El registro del Pi que avisa de bajo voltaje o recorte.                             |
| Headless              | Sin monitor ni teclado: todo por SSH.                                               |
| Régimen               | El comportamiento estable, pasado el arranque.                                      |

### Qué es

Un robot de dos ruedas que busca un oso impreso y huye de una mochila, decidiendo todo a bordo con una red neuronal que corre a 11 cuadros por segundo en una Raspberry Pi 5.

### Cómo está hecho

Dos niveles: el Pi ve y decide; el ESP32 mueve y protege. Entre ellos, un protocolo de texto. Todos los números en un archivo, cada uno con su origen. El comportamiento, verificable sin robot.

### Dónde está

Funciona en el piso: busca, encuentra, se acerca, frena a 30 cm y huye. Falta medirlo con series de corridas, blindarlo para la sala (red, imagen rota) y escribir el informe.

### Lo que enseñó

Que un número vale en el contexto donde se midió. Veintiún veces, desde la caída de tensión de un integrado hasta la etiqueta que una red le pone a una mochila vista desde el piso.